InsightInk

راهنمای جامع معماری، یادگیری و توسعه

Developer Bible

پروژه کارشناسی

نسخه برنامه‌ریزی اولیه – آبان ۱۴۰۵ / اکتبر ۲۰۲۶

وضعیت سند

این سند نقشه اصلی توسعه پروژه InsightInk است. هدف آن این است که توسعه‌دهنده بتواند حتی بدون دسترسی به ابزارهای هوش مصنوعی، مسیر پروژه را از مرحله آماده‌سازی محیط تا پیاده‌سازی، تست، استقرار و مستندسازی دانشگاهی ادامه دهد.
نسخه‌های محلی تأییدشده هنگام نگارش:

- Python 3.14.8
- Django 5.2.18
- سیستم توسعه: Linux / PyCharm / Git

این سند بین سه نوع اطلاعات تفکیک می‌کند:

۱. نیازمندی‌های مستقیم استخراج‌شده از PDS، بردهای Epic/User Story/NFR/WSJF و پروتوتایپ؛
۲. تصمیم‌های معماری و محدودسازی دامنه که بعداً برای پروژه کارشناسی گرفته شده‌اند؛
۳. پیشنهادهای فنی و آموزشی که برای اجرای قابل‌فهم و قابل‌دفاع پروژه اضافه شده‌اند.

فهرست مطالب

چگونه از این سند استفاده کنیم **ط**

۱ معرفی محصول و مسئله **۱**

۱.۱ InsightInk چیست؟ ۱

۱.۲ هدف این پروژه دانشگاهی ۱

۱.۳ هایActor اصلی ۲

۱.۳.۱ User / Reader ۲

۱.۳.۲ Creator ۲

۱.۳.۳ Moderator ۲

۱.۳.۴ Admin ۳

۱.۴ زبان‌های محصول ۳

۱.۵ تعریف AI در این پروژه ۳

۱.۶ معیار موفقیت پروژه ۴

۲ دامنه پروژه و نیازمندی‌ها **۵**

۲.۱ منابع نیازمندی ۵

۲.۲ قابلیت‌های اصلی برگرفته از PDS ۵

۲.۳ نه Epic اصلی ۶

۲.۴ تفکیک Scope نهایی ۶

۲.۴.۱ MVP Core ۶

۲.۴.۲ فاز دوم ۷

۲.۴.۳ موارد خارج از Scope ۷

۲.۵ هایNFR منبع ۷

۶.۲ تغییر تفسیر NFR-۰۹ . . . . . ۸
۷.۲ تغییر تفسیر NFR-۱۷ . . . . . ۹
۸.۲ منبع WSJF ۹
۹.۲ تعریف Done برای MVP Core ۹

۳ فلسفه توسعه و روش یادگیری ۱۱

۱.۳ اصل اول: پروژه باید قابل فهم بماند ۱۱
۲.۳ اصل دوم: Magic Before Learn ۱۱
۳.۳ اصل سوم: از دانسته‌های Node.js استفاده می‌کنیم ۱۲
۴.۳ اصل چهارم: کوچک‌ترین سیستم درست ۱۲
۵.۳ اصل پنجم: Truth of Source Database ۱۳
۶.۳ اصل ششم: هر Feature شش حالت دارد ۱۳
۷.۳ اصل هفتم: Done of Definition واقعی ۱۳
۸.۳ اصل هشتم: هر Commit یک مفهوم ۱۳
۹.۳ اصل نهم: Performance قبل از Optimization کور ۱۴
۱۰.۳ اصل دهم: Journal Project ۱۴

۴ معماری کلان سیستم ۱۵

۱.۴ سبک معماری ۱۵
۲.۴ نمودار Baseline ۱۶
۳.۴ نمودار فاز Realtime ۱۶
۴.۴ مرزهای Backend ۱۷
۱.۴.۴ accounts ۱۷
۲.۴.۴ content ۱۷
۳.۴.۴ creators ۱۸
۴.۴.۴ engagement ۱۸
۵.۴.۴ library ۱۸
۶.۴.۴ recommendations ۱۸
۷.۴.۴ moderation ۱۸
۸.۴.۴ messaging و notifications ۱۹

۵.۴ چرا Layer Service محدود استفاده می‌کنیم؟ ۱۹
۶.۴ اصل Direction Dependency ۱۹
۷.۴ Search ۲۰
۸.۴ Media ۲۰
۹.۴ اصل Degradation Graceful ۲۰

۵ Stack Technology و دلیل انتخاب هر ابزار ۲۱

۱.۵ Backend ۲۱
۱.۱.۵ Python ۸.۱۴.۳ . . . . . ۲۱
۲.۱.۵ Django ۱۸.۲.۵ . . . . . ۲۱
۳.۱.۵ Framework REST Django ۲۲
۴.۱.۵ PostgreSQL ۲۲
۵.۱.۵ Redis - فقط بعداً ۲۲
۶.۱.۵ Channels Django - فقط بعداً ۲۳
۷.۱.۵ Celery - فعلاً خیر ۲۳
۲.۵ Frontend ۲۳
۱.۲.۵ HTML ۲۳
۲.۲.۵ CSS ۲۴
۳.۲.۵ JavaScript ۲۴
۴.۲.۵ TypeScript ۲۴
۵.۲.۵ React ۲۵
۶.۲.۵ Vite ۲۵
۷.۲.۵ Router React ۲۵
۸.۲.۵ Query TanStack ۲۶
۹.۲.۵ next۱۸react-i / next۱۸i ۲۶
۳.۵ Stack Testing ۲۶
۱.۳.۵ Backend ۲۶
۲.۳.۵ Frontend ۲۷
۳.۳.۵ End-to-End ۲۷
۴.۳.۵ Test Load ۲۷

۴.۵ Quality Code ۲۷
۵.۵ ابزارهایی که انتخاب نشده‌اند ۲۷
۶.۵ قاعده انتخاب Dependency ۲۸

۶ ساختار Repository و آماده‌سازی محیط ۲۹
۱.۶ وضعیت فعلی سیستم ۲۹
۲.۶ ساختار نهایی Monorepo ۲۹
۳.۶ Environment Virtual ۳۰
۴.۶ Cleanup Git قبل از کدنویسی ۳۰
۵.۶ Gitignore پیشنهادی ۳۱
۶.۶ ساخت Backend ۳۱
۷.۶ ساخت Frontend ۳۲
۸.۶ PostgreSQL ۳۲
۹.۶ Variables Environment ۳۳
۱۰.۶ Files Dependency ۳۳
۱۱.۶ روال روزانه توسعه ۳۳
۱۲.۶ Done of Definition برای Phase ۰ . . . . . ۳۴

۷ مسیر یادگیری Python و Django ۳۵
۱.۷ هدف این فصل ۳۵
۲.۷ Python: چیزهایی که واقعاً لازم داریم ۳۵
۱.۲.۷ مرحله ۱P: Syntax و Types Data ۳۵
۲.۲.۷ مرحله ۲P: Function ۳۶
۳.۲.۷ مرحله ۳P: Module و Package ۳۶
۴.۲.۷ مرحله ۴P: Class و Object ۳۶
۵.۲.۷ مرحله ۵P: Exception ۳۷
۶.۲.۷ مرحله ۶P: Comprehension و Iterator ۳۷
۳.۷ Django از چه اجزایی تشکیل شده؟ ۳۷
۱.۳.۷ Project ۳۷
۲.۳.۷ App ۳۸

۴.۷ Routing URL ۳۸
۵.۷ Request و Response ۳۸
۶.۷ Model و ORM ۳۸
۷.۷ Migration ۳۹
۸.۷ Admin Django ۳۹
۹.۷ User Custom از روز اول ۴۰
۱۰.۷ Authentication Django ۴۰
۱۱.۷ DRF بعد از Django پایه ۴۰
۱۲.۷ ترتیب تمرین‌های Backend ۴۱
۱۳.۷ معیار فهم ۴۱

۸ طراحی Backend و ساختار Django ۴۳

۱.۸ ساختار پیشنهادی ۴۳
۲.۸ config ۴۳
۳.۸ accounts ۴۴
۴.۸ content ۴۴
۵.۸ creators ۴۵
۶.۸ engagement ۴۵
۷.۸ library ۴۵
۸.۸ recommendations ۴۵
۹.۸ moderation ۴۶
۱۰.۸ Logic Business کجا قرار بگیرد؟ ۴۶
۱.۱۰.۸ Model ۴۶
۲.۱۰.۸ Serializer ۴۶
۳.۱۰.۸ View ۴۶
۴.۱۰.۸ Service ۴۶
۵.۱۰.۸ helper Selector/Query ۴۶
۱۱.۸ Transaction ۴۷
۱۲.۸ Signals ۴۷
۱۳.۸ Settings ۴۷

۱۴.۸ Logging ۴۷
۱۵.۸ strategy Admin ۴۸
۱۶.۸ data Seed ۴۸
۱۷.۸ Command Management ۴۸
۱۸.۸ Done of Definition Backend ۴۹

۹ طراحی Database و مدل داده ۵۱

۱.۹ اصل طراحی ۵۱
۲.۹ User ۵۱
۳.۹ CreatorProfile ۵۱
۴.۹ Topic ۵۲
۵.۹ UserInterest ۵۲
۶.۹ Source ۵۲
۷.۹ Idea ۵۳
۸.۹ Spark ۵۳
۹.۹ SavedIdea ۵۴
۱۰.۹ Comment ۵۴
۱۱.۹ CreatorFollow ۵۴
۱۲.۹ ReadingEvent ۵۴
۱۳.۹ Report ۵۵
۱۴.۹ ModerationAction ۵۵
۱۵.۹ later - Messaging ۵۶
۱۶.۹ later - Notification ۵۶
۱۷.۹ later - Membership/eBook ۵۶
۱۸.۹ ERD مفهومی ۵۷
۱۹.۹ Strategy Index ۵۷
۲۰.۹ Normalization ۵۸
۲۱.۹ Rules Migration ۵۸

۱۰ Authentication API، REST و Roleها ۵۹

۱.۱۰ Base API ۵۹
۲.۱۰ HTTP هایMethod ۵۹
۳.۱۰ هایCore Endpoint پیشنهادی ۶۰
۱.۳.۱۰ Account ۶۰
۲.۳.۱۰ Feed/Idea ۶۰
۳.۳.۱۰ Engagement ۶۰
۴.۳.۱۰ Library ۶۰
۵.۳.۱۰ Creator ۶۱
۶.۳.۱۰ Search ۶۱
۷.۳.۱۰ Recommendation ۶۱
۴.۱۰ Pagination ۶۱
۵.۱۰ Authentication ۶۲
۶.۱۰ Session یا Token؟ ۶۲
۷.۱۰ Authorization ۶۲
۸.۱۰ Reader ۶۲
۹.۱۰ Creator ۶۳
۱۰.۱۰ Moderator ۶۳
۱۱.۱۰ Admin ۶۳
۱۲.۱۰ Strategy Group/Permission ۶۴
۱۳.۱۰ Example Permission Object-Level ۶۴
۱۴.۱۰ Format Error Validation ۶۴
۱۵.۱۰ OpenAPI ۶۴
۱۶.۱۰ Done of Definition API ۶۵

۱۱ مسیر یادگیری Frontend از صفر ۶۷

۱.۱۱ چرا مستقیم وارد React نمی‌شویم؟ ۶۷
۲.۱۱ مرحله ۱F: HTML ۶۷
۳.۱۱ مرحله ۲F: CSS ۶۸
۴.۱۱ مرحله ۳F: JavaScript Browser ۶۸
۱.۴.۱۱ DOM ۶۸

۲.۴.۱۱ Fetch ۶۹
۵.۱۱ مرحله ۴F: npm و package.json ۶۹
۶.۱۱ مرحله ۵F: TypeScript ۷۰
۷.۱۱ مرحله ۶F: Fundamentals React ۷۱
۱.۷.۱۱ Component ۷۱
۲.۷.۱۱ Props ۷۱
۳.۷.۱۱ State ۷۱
۴.۷.۱۱ Event ۷۱
۵.۷.۱۱ Rendering List ۷۱
۸.۱۱ مرحله ۷F: Hooks ۷۲
۹.۱۱ مرحله ۸F: Routing ۷۲
۱۰.۱۱ مرحله ۹F: State Server ۷۳
۱۱.۱۱ تمرین‌های Frontend در ترتیب واقعی ۷۳
۱۲.۱۱ معیار فهم Frontend ۷۳

۱۲ طراحی Frontend و تبدیل Prototype به محصول واقعی ۷۵

۱.۱۲ Prototype چه چیزی به ما می‌دهد؟ ۷۵
۲.۱۲ Design هایToken قابل انتقال ۷۶
۳.۱۲ Structure Theme ۷۶
۴.۱۲ هایComponent اصلی ۷۶
۱.۴.۱۲ Primitive/Shared ۷۶
۲.۴.۱۲ Components Domain ۷۷
۵.۱۲ folders Feature-based ۷۷
۶.۱۲ Component vs Page ۷۸
۷.۱۲ Design Feed ۷۸
۸.۱۲ Contract Data IdeaCard ۷۸
۹.۱۲ UI Optimistic ۷۹
۱۰.۱۲ Forms ۷۹
۱۱.۱۲ Client API ۸۰
۱۲.۱۲ Strategy Responsive ۸۰

۱۳.۱۲ Rules UI Device Old ۸۰
۱۴.۱۲ Mapping Screen Prototype ۸۱
۱۵.۱۲ Done of Definition Frontend ۸۱

۱۳ سیستم پیشنهاددهی الگوریتمی ۸۳

۱.۱۳ هدف ۸۳
۲.۱۳ Start Cold ۸۳
۳.۱۳ هایSignal اصلی ۸۴
۴.۱۳ فرمول پایه ۸۴
۵.۱۳ جلوگیری از رشد بی‌نهایت Affinity ۸۵
۶.۱۳ Similarity Jaccard ۸۵
۷.۱۳ Diversity Cross-format ۸۶
۸.۱۳ Popularity ۸۶
۹.۱۳ Freshness ۸۶
۱۰.۱۳ Explanation Recommendation ۸۶
۱۱.۱۳ Ranking Feed و Idea Related جدا هستند ۸۷
۱.۱۱.۱۳ Feed ۸۷
۲.۱۱.۱۳ Related ۸۷
۱۲.۱۳ ساختار کد پیشنهادی ۸۷
۱۳.۱۳ Function Pure ۸۷
۱۴.۱۳ Performance ۸۸
۱۵.۱۳ Evaluation ۸۸
۱۶.۱۳ TF-IDF به‌عنوان Stretch ۸۹
۱۷.۱۳ Conflict با s۳۰ in Summarize ۸۹

۱۴ Realtime، Messaging Sync و Notifications ۹۱

۱.۱۴ چرا این بخش بعداً می‌آید؟ ۹۱
۲.۱۴ Sync Cross-Device ۹۱
۳.۱۴ Flow صحیح Write ۹۲
۴.۱۴ Channels و Redis ۹۲

۱۴.۵ Lifecycle WebSocket ۹۲
۱۴.۶ Indicator Sync ۹۳
۱۴.۷ Scope Messaging ۹۳
۱۴.۸ Flow Message ۹۳
۱۴.۹ Authorization Conversation ۹۳
۱۴.۱۰ Indicator Typing ۹۴
۱۴.۱۱ Receipt Read ۹۴
۱۴.۱۲ Block و Report ۹۴
۱۴.۱۳ Notifications ۹۴
۱۴.۱۴ Notification In-App اول ۹۵
۱۴.۱۵ Preference Notification ۹۵
۱۴.۱۶ Batching ۹۵
۱۴.۱۷ Test Realtime ۹۵
۱۴.۱۸ Scenarios Failure ۹۶
۱۴.۱۹ تعریف Realtime Done ۹۶

۱۵ PWA TTS، و سازگاری با گوشی‌های قدیمی ۹۷

۱۵.۱ اصل سازگاری ۹۷
۱۵.۲ Tier بندی هاFeature ۹۷
۱۵.۲.۱ Core :۱ Tier ۹۷
۱۵.۲.۲ Enhancement :۲ Tier ۹۷
۱۵.۳ چرا ؟Web/PWA-first ۹۸
۱۵.۴ PWA چیست؟ ۹۸
۱۵.۵ Order Implementation PWA ۹۸
۱۵.۶ Requirement TTS ۹۹
۱۵.۷ TTS گزینه :Speech Browser/Device A ۹۹
۱۵.۸ TTS گزینه :Audio Pre-generated B ۹۹
۱۵.۹ تصمیم TTS ۱۰۰
۱۵.۱۰ Target اولیه Browser ۱۰۰
۱۵.۱۱ Performance Frontend روی Device ضعیف ۱۰۰

۱۲.۱۵ Build Legacy ۱۰۱
۱۳.۱۵ Offline ۱۰۱
۱۴.۱۵ Matrix Compatibility ۱۰۱

۱۶ بین‌المللی‌سازی، RTL/LTR و Accessibility ۱۰۳
۱.۱۶ زبان‌ها ۱۰۳
۲.۱۶ Language Content vs Language UI ۱۰۳
۳.۱۶ Files Translation ۱۰۳
۴.۱۶ text Hard-coded ۱۰۴
۵.۱۶ Direction ۱۰۴
۶.۱۶ Properties Logical CSS ۱۰۴
۷.۱۶ Normalization Text Persian ۱۰۵
۸.۱۶ Target Accessibility ۱۰۵
۹.۱۶ HTML Semantic ۱۰۵
۱۰.۱۶ Keyboard ۱۰۶
۱۱.۱۶ Reader Screen ۱۰۶
۱۲.۱۶ Color/Contrast ۱۰۶
۱۳.۱۶ Targets Touch ۱۰۶
۱۴.۱۶ TTS مستقل از Accessibility ۱۰۷
۱۵.۱۶ Testing ۱۰۷
۱۶.۱۶ Done of Definition Bilingual ۱۰۷

۱۷ امنیت، حریم خصوصی و Moderation ۱۰۹
۱.۱۷ اصل کلی ۱۰۹
۲.۱۷ Password ۱۰۹
۳.۱۷ Authorization vs Authentication ۱۰۹
۴.۱۷ Ownership Object ۱۱۰
۵.۱۷ CSRF ۱۱۰
۶.۱۷ CORS ۱۱۰
۷.۱۷ XSS ۱۱۰

۱۱۰ . . . . . Upload File ۸.۱۷
۱۱۱ . . . . . Data Privacy ۹.۱۷
۱۱۱ . . . . . Account Delete ۱۰.۱۷
۱۱۱ . . . . . Privacy Message Direct ۱۱.۱۷
۱۱۲ . . . . . Payment ۱۲.۱۷
۱۱۲ . . . . . Report ۱۳.۱۷
۱۱۲ . . . . . Workflow Moderation ۱۴.۱۷
۱۱۳ . . . . . Audit ModerationAction ۱۵.۱۷
۱۱۳ . . . . . Assignment Moderator ۱۶.۱۷
۱۱۳ . . . . . Block ۱۷.۱۷
۱۱۳ . . . . . Mute ۱۸.۱۷
۱۱۳ . . . . . Limiting Rate ۱۹.۱۷
۱۱۴ . . . . . Secrets ۲۰.۱۷
۱۱۴ . . . . . Logging ۲۱.۱۷
۱۱۴ . . . . . Feature Checklist Security ۲۲.۱۷

۱۱۵ . . . . . ۱۸ Quality Testing، سنجش هاNFR و

۱۱۵ . . . . . Pyramid Test ۱.۱۸
۱۱۵ . . . . . Test Unit Backend ۲.۱۸
۱۱۶ . . . . . Test Model/DB ۳.۱۸
۱۱۶ . . . . . Test API ۴.۱۸
۱۱۶ . . . . . Test Frontend ۵.۱۸
۱۱۶ . . . . . E۲E ۶.۱۸
۱۱۷ . . . . . ۰۱NFR- ۷.۱۸
۱۱۷ . . . . . ۰۲NFR- ۸.۱۸
۱۱۸ . . . . . ۰۳NFR- ۹.۱۸
۱۱۸ . . . . . ۰۴NFR- ۱۰.۱۸
۱۱۸ . . . . . ۰۵NFR- ۱۱.۱۸
۱۱۸ . . . . . ۰۶NFR- ۱۲.۱۸
۱۱۹ . . . . . ۰۷NFR- ۱۳.۱۸

۱۴.۱۸ NFR-۰۸ . . . . . ۱۱۹
۱۵.۱۸ NFR-۱۶ . . . . . ۱۱۹
۱۶.۱۸ NFR-۱۷ . . . . . ۱۱۹
۱۷.۱۸ Data Test ۱۱۹
۱۸.۱۸ CI ۱۲۰
۱۹.۱۸ Coverage ۱۲۰
۲۰.۱۸ Gate Quality ۱۲۰

۱۹ Git، Branching Workflow و نظم توسعه ۱۲۱
۱.۱۹ هدف ۱۲۱
۲.۱۹ Phase ۰ Cleanup ۱۲۱
۳.۱۹ Strategy Branch ۱۲۱
۴.۱۹ Message Commit ۱۲۲
۵.۱۹ قبل از Commit ۱۲۲
۶.۱۹ هایCommit بد ۱۲۳
۷.۱۹ Migration و Commit ۱۲۳
۸.۱۹ Lockfile ۱۲۳
۹.۱۹ Check Secrets ۱۲۳
۱۰.۱۹ هایMilestone Tag ۱۲۳
۱۱.۱۹ Journal Project ۱۲۴
۱۲.۱۹ Mindset Recovery ۱۲۴

۲۰ Roadmap اجرایی کامل پروژه ۱۲۵
۱.۲۰ قاعده Roadmap ۱۲۵
۲.۲۰ Phase ۰: Foundation Repository ۱۲۵
۳.۲۰ Phase ۱: Foundation Django ۱۲۶
۴.۲۰ Phase ۲: Models Content Core ۱۲۶
۵.۲۰ Phase ۳: Foundation API REST ۱۲۷
۶.۲۰ Phase ۴: Roles and Authentication ۱۲۷
۷.۲۰ Phase ۵: Fundamentals Frontend ۱۲۸

۸.۲۰ Phase ۶: Foundation React ۱۲۸
۹.۲۰ Phase ۷: Onboarding ۱۲۸
۱۰.۲۰ Phase ۸: Feed Core ۱۲۹
۱۱.۲۰ Phase ۹: Engagement ۱۲۹
۱۲.۲۰ Phase ۱۰: Library Personal ۱۲۹
۱۳.۲۰ Phase ۱۱: Network Creator ۱۲۹
۱۴.۲۰ Phase ۱۲: Search and Explore ۱۳۰
۱۵.۲۰ Phase ۱۳: Algorithm Recommendation ۱۳۰
۱۶.۲۰ Phase ۱۴: Sync Realtime ۱۳۰
۱۷.۲۰ Phase ۱۵: Messaging ۱۳۰
۱۸.۲۰ Phase ۱۶: Notifications ۱۳۱
۱۹.۲۰ Phase ۱۷: TTS ۱۳۱
۲۰.۲۰ Phase ۱۸: Membership/eBooks ۱۳۱
۲۱.۲۰ Phase ۱۹: Completion Moderation ۱۳۱
۲۲.۲۰ Phase ۲۰: Hardening ۱۳۲
۲۳.۲۰ Phase ۲۱: Deployment ۱۳۲
۲۴.۲۰ Phase ۲۲: Package Academic Final ۱۳۲
۲۵.۲۰ Dependency Rules ۱۳۳

۲۱ Playbookهای پیاده‌سازی Feature ۱۳۵

۱.۲۱ Playbook ساخت Model جدید ۱۳۵
۲.۲۱ Playbook ساخت API جدید ۱۳۶
۳.۲۱ Playbook ساخت Screen React ۱۳۶
۴.۲۱ Playbook Mutation مثل Save ۱۳۷
۵.۲۱ Playbook Pagination ۱۳۷
۶.۲۱ Playbook Permission ۱۳۷
۷.۲۱ Playbook Recommendation تغییر Weight ۱۳۸
۸.۲۱ Playbook Bug ۱۳۸
۹.۲۱ Playbook Performance Database ۱۳۸
۱۰.۲۱ Playbook Dependency ۱۳۹

۱۱.۲۱ Refactor Playbook ۱۳۹
۱۲.۲۱ Playbook وقتی گیر کردی ۱۳۹

۲۲ Deployment و Readiness Production ۱۴۱
۱.۲۲ هدف ۱۴۱
۲.۲۲ های Production Component ۱۴۱
۳.۲۲ Build Frontend ۱۴۱
۴.۲۲ Settings Production Django ۱۴۲
۵.۲۲ ASGI vs WSGI ۱۴۲
۶.۲۲ Static و Media ۱۴۲
۷.۲۲ Deployment Migration Database ۱۴۲
۸.۲۲ Backup ۱۴۳
۹.۲۲ Logging ۱۴۳
۱۰.۲۲ Monitoring ۱۴۳
۱۱.۲۲ CDN ۱۴۳
۱۲.۲۲ Path Scaling ۱۴۴
۱۳.۲۲ Checklist Release ۱۴۴

۲۳ ردیابی کامل نیازمندی‌های منبع ۱۴۵
۱.۲۳ هدف ۱۴۵
۲.۲۳ Epic: Discovery Idea AI-Powered ۱۴۵
۳.۲۳ Epic: Experience Cross-Device Real-Time، ۱۴۶
۴.۲۳ Epic: Network Direct-Access Creator ۱۴۶
۵.۲۳ Epic: eBooks & Membership Library ۱۴۶
۶.۲۳ Epic: Content Multi-Format & Feed Idea Core ۱۴۶
۷.۲۳ Epic: Setup Account & Onboarding ۱۴۷
۸.۲۳ Epic: Signals Social & Engagement ۱۴۷
۹.۲۳ Epic: Access Multimodal (Text-to-Speech) ۱۴۷
۱۰.۲۳ Epic: Awareness Activity & Notifications ۱۴۷
۱۱.۲۳ جدول NFR Treatment ۱۴۷

۱۲.۲۳ WSJF کامل ۱۴۸
۱۳.۲۳ Matrix Traceability پیشنهادی برای پایان پروژه ۱۴۹

۲۴ Cheat Sheet، تصمیم‌های باز و راهنمای ادامه بدون AI **۱۵۱**

۱.۲۴ های Command پایه Git ۱۵۱
۲.۲۴ Python/Virtualenv ۱۵۱
۳.۲۴ Django ۱۵۱
۴.۲۴ Frontend ۱۵۲
۵.۲۴ Checklist Conceptual PostgreSQL ۱۵۲
۶.۲۴ Reference Quick HTTP ۱۵۳
۷.۲۴ Backend Checklist Debugging ۱۵۳
۸.۲۴ Frontend Checklist Debugging ۱۵۳
۹.۲۴ Name Repository Decision: Open ۱۵۴
۱۰.۲۴ Branch Git Decision: Open ۱۵۴
۱۱.۲۴ Management Dependency Python Decision: Open ۱۵۴
۱۲.۲۴ Login Social Decision: Open ۱۵۴
۱۳.۲۴ Definition Read Decision: Open ۱۵۵
۱۴.۲۴ TTS Decision: Open ۱۵۵
۱۵.۲۴ Payment/eBooks Decision: Open ۱۵۵
۱۶.۲۴ Push Web Decision: Open ۱۵۵
۱۷.۲۴ وقتی AI در دسترس نیست ۱۵۵
۱۸.۲۴ قاعده مطالعه Documentation ۱۵۶
۱۹.۲۴ چک‌لیست شروع هر Feature ۱۵۶
۲۰.۲۴ چک‌لیست پایان هر Session ۱۵۶
۲۱.۲۴ Deliverables Project Final ۱۵۶
۲۲.۲۴ آخرین اصل ۱۵۷

جمع‌بندی **۱۵۹**

چگونه از این سند استفاده کنیم

این سند قرار نیست فقط خوانده شود. در طول توسعه باید نقش چک‌لیست و مرجع روزمره را داشته باشد.

چرخه پیشنهادی هر مرحله:

یادگیری ← فهم ← پیاده‌سازی ← تست ← مستندسازی ← Commit

اگر در هر مرحله نتوانی توضیح بدهی یک ابزار چه مشکلی را حل می‌کند، چرا به پروژه اضافه شده، چه داده‌ای می‌گیرد و چه خروجی‌ای می‌دهد، هنوز زمان عبور از آن مرحله نیست.

فصل ۱

معرفی محصول و مسئله

۱.۱ InsightInk چیست؟

InsightInk یک سامانه محتوایی است که ایده‌های مفید استخراج‌شده از کتاب، مقاله، فیلم/مستند و پادکست را در قالب یک Feed در اختیار کاربر قرار می‌دهد. ایده اصلی محصول این است که «اسکرول کردن» به‌جای مصرف محتوای صرفاً سرگرم‌کننده، به مواجهه با تجربه‌ها و ایده‌های قابل‌فکر و قابل‌عمل تبدیل شود.

برگرفته از نیازمندی‌های منبع

در PDS، محصول به‌عنوان فضایی برای مرور ایده‌های حاصل از تجربه خواندن کتاب و مقاله، دیدن فیلم و شنیدن پادکست معرفی شده است. کاربر می‌تواند از این محتوا الهام بگیرد، آن را ذخیره یا به اشتراک بگذارد و با سازندگان محتوا ارتباط بگیرد.

۲.۱ هدف این پروژه دانشگاهی

هدف ما ساخت یک استارتاپ production در مقیاس میلیون‌ها کاربر نیست. هدف عبارت است از:

- ساخت یک پروژه Full-Stack واقعی و قابل اجرا؛
- یادگیری عمیق Python و Django؛
- یادگیری Frontend از مبانی تا React + TypeScript؛
- طراحی درست دیتابیس رابطه‌ای؛
- ساخت API، احراز هویت، Permission و تست؛
- پیاده‌سازی سیستم پیشنهاددهی الگوریتمی قابل توضیح؛
- پشتیبانی هم‌زمان از فارسی و انگلیسی؛
- داشتن مستندات کامل برای ادامه کار بدون وابستگی به AI.

تصمیم پروژه

معماری باید برای یک نفر قابل نگهداری باشد. بنابراین microservice، Kubernetes، event bus، vector database و زیرساخت‌های سنگین وارد Baseline نمی‌شوند.

۳.۱ های Actor اصلی

۱.۳.۱ User / Reader

کاربر معمولی که می‌تواند:

- ایده‌ها را بخواند؛
- Spark بزند؛
- ذخیره کند؛
- نظر بدهد؛
- Creator را Follow کند؛
- ایده‌ها را جست‌وجو کند؛
- کتابخانه شخصی داشته باشد؛
- محتوای نامناسب را Report کند؛
- در فاز پیام‌رسانی با Creator تماس بگیرد.

۲.۳.۱ Creator

Creator یک User جداگانه از نظر هویتی نیست؛ یک User است که CreatorProfile و های Permission مربوط را دارد. او می‌تواند پروفایل سازنده داشته باشد، محتوای خودش را منتشر/مدیریت کند و با مخاطب تعامل داشته باشد.

۳.۳.۱ Moderator

Moderator ناظر سیستم است و دسترسی‌های محدود اما قوی برای Safety & Trust دارد:

- مشاهده ها؛Report
- بررسی محتوای گزارش‌شده؛
- مخفی/حذف کردن Comment یا Content طبق سیاست؛
- ثبت اقدام ؛Moderation
- اعمال محدودیت‌های مشخص روی User در صورت نیاز.

Moderator مدیر کل نیست و اجازه ایجاد Moderator یا Admin ندارد.

۴.۳.۱ Admin

Admin مدیر کل سیستم است و بالاترین دسترسی را دارد. فقط Admin می‌تواند هاModerator را تعیین یا حذف کند.

۴.۱ زبان‌های محصول

تصمیم پروژه

زبان اصلی محصول فارسی و زبان دوم انگلیسی است. از روز اول ساختار Frontend و مدل داده باید امکان RTL/LTR و محتوای دو زبانه را در نظر بگیرد.

این یعنی:

- UI فارسی راست‌به‌چپ؛
- UI انگلیسی چپ‌به‌راست؛
- زبان UI از زبان Content جداست؛
- کاربر می‌تواند UI فارسی داشته باشد و محتوای انگلیسی ببیند.

۵.۱ تعریف AI در این پروژه

هشدار / دام رایج

در منابع اولیه عباراتی مانند AI-Assisted Tool و AI-Powered Idea Discovery آمده است. در Scope نهایی این پروژه، هیچ LLM یا RAG یا Agent وجود ندارد.

سیستم پیشنهاددهی ما کاملاً الگوریتمی است و با داده‌هایی مانند Interest، Save، Spark، Follow تازگی محتوا و Popularity کار می‌کند.

مواردی که صریحاً خارج از پروژه هستند:

- OpenAI/Claude/Gemini API
- LLM
- RAG
- Embeddings

- Vector DB
- Prompt Engineering
- AI Agent

۶.۱ معیار موفقیت پروژه

در پایان، پروژه باید حداقل بتواند Flow زیر را واقعاً اجرا کند:

۱. کاربر Account بسازد و Login کند؛
۲. زبان و علاقه‌مندی‌ها را انتخاب کند؛
۳. Feed واقعی از PostgreSQL ببیند؛
۴. Idea را باز کند؛
۵. Source آن را ببیند؛
۶. Spark/Save/Comment کند؛
۷. Saved هاIdea را در Library ببیند؛
۸. Creator را ببیند و Follow کند؛
۹. Search/Explore داشته باشد؛
۱۰. Recommendation الگوریتمی دریافت کند؛
۱۱. UI فارسی و انگلیسی درست داشته باشد.

Realtime، Messaging، Notification TTS و Membership/eBook بعد از این Flow Core اضافه می‌شوند.

فصل ۲

دامنه پروژه و نیازمندی‌ها

۱.۲ منابع نیازمندی

مبنای این سند مجموعه فایل‌های پروژه است:

- PDS.pdf
- بردهای Epic / Feature / User Story
- برد Non-Functional Requirements
- برد WSJF Prioritization
- پروتوتایپ InsightInk Investor Prototype

پروتوتایپ به‌عنوان مرجع Language Design Flow، و هاScreen استفاده می‌شود، نه pro- Codebase duction.

۲.۲ قابلیت‌های اصلی برگرفته از PDS

برگرفته از نیازمندی‌های منبع

PDS قابلیت‌های زیر را مشخص می‌کند:

- خواندن یا Share کردن تجربه مرتبط با کتاب، مقاله، فیلم یا پادکست؛
- Share Save، و Comment روی Idea؛
- Pagination؛
- Time؛ Reading Estimated
- Text-To-Speech؛
- Algorithm؛ Exploring Idea و Suggesting Idea

- eBook Library، و Membership؛
- ارتباط مستقیم با Maker؛ Writer/Director/Podcast
- relational SQL، CDN database و real-time sync.

۳.۲ نه Epic اصلی

۱. AI-Powered Idea Discovery
۲. Real-Time, Cross-Device Experience
۳. Creator Direct-Access Network
۴. Library Membership & eBooks
۵. Core Idea Feed & Multi-Format Content
۶. Onboarding & Account Setup
۷. Engagement & Social Signals
۸. Multimodal Access (Text-to-Speech)
۹. Notifications & Activity Awareness

۴.۲ تفکیک Scope نهایی

۱.۴.۲ MVP Core

MVP Core باید شامل این موارد باشد:

- Register/Login؛
- Profile؛
- انتخاب Interest؛
- model؛ data Topic/Source/Idea/Creator
- Feed با Pagination؛
- Detail؛ Idea
- Search/Explore؛
- Spark؛
- Save/Library؛
- Comment؛
- Profile؛ Creator
- Follow؛

- Report پایه؛
- Recommendation الگوریتمی؛
- فارسی/انگلیسی و RTL/LTR.

۲.۴.۲ فاز دوم

بعد از Core:

- Sync؛ Realtime
- Messaging؛ Direct
- Notifications؛
- TTS؛
- Membership/eBooks؛
- behavior Offline در حدی که واقعاً قابل انجام و قابل تست باشد.

۳.۴.۲ موارد خارج از Scope

- AI؛ Generative
- RAG؛
- Embedding؛
- Agent؛
- Microservice؛
- codebase mobile Native جدا؛
- ذخیره اطلاعات کارت بانکی؛
- ادعای SLA واقعی تجاری بدون داده.

۵.۲ منبع NFRهای

ID	دسته	نیازمندی منبع
NFR-۰۱	Performance	تمام تعاملات UI باید زیر ۱۰۰ms پاسخ ادراکی داشته باشند.
NFR-۰۲	Performance	صفحه بعد Feed در شرایط معمول شبکه زیر ۳۰۰ms دریافت شود.

ID	دسته	نیازمندی منبع
NFR-۰۳	Scalability	Backend بتواند هزاران کاربر هم‌زمان برای scroll/save/message را پشتیبانی کند.
NFR-۰۴	Scalability	زیرساخت قابلیت scale افقی/elastic داشته باشد.
NFR-۰۵	Reliability	change Save/Library حداکثر ظرف ۲ ثانیه بین De-هاsync vice شود.
NFR-۰۶	Reliability	هدف ۹۹.۹% uptime.
NFR-۰۷	Efficiency	مصرف CPU/RAM پس‌زمینه پایین باشد.
NFR-۰۸	Efficiency	Media از CDN سریع سرو شود.
NFR-۰۹	Compatibility	منبع: iOS native ۱۴+ و Android ۹+.
NFR-۱۰	Compatibility	Web به‌صورت PWA. installable
NFR-۱۱	Data	data Core در DB. relational SQL
NFR-۱۲	Privacy	Reading history/saves/personalization قابلیت export/delete داشته باشد.
NFR-۱۳	Security	Message Direct در transit رمزگذاری و فقط برای Participantها قابل دسترسی باشد.
NFR-۱۴	Payment	data card Raw روی سرور ما ذخیره نشود.
NFR-۱۵	Safety	Report/Block/Mute وجود داشته باشد.
NFR-۱۶	Accessibility	WCAG ۱.۲ AA مستقل از TTS.
NFR-۱۷	Quality AI	پیشنهادها زیر ۲ ثانیه.
NFR-۱۸	i۱۸n	discovery UI/Content/Creator چندزبانه.

۶.۲ تغییر تفسیر NFR-۰۹

تصمیم پروژه

به دلیل اهمیت گوشی‌های قدیمی، Delivery اصلی Web/PWA-first است. بنابراین نیازمندی iOS/Android native به‌صورت مستقیم پیاده نمی‌شود. هدف این است که Core تجربه روی هایBrowser قدیمی‌تر کار کند و هایFeature غیرضروری Degradation Graceful داشته باشند.

Target اولیه برای تست می‌تواند حوالی iOS ۱۲+ و Android ۷+ باشد، اما این یک Target Test است نه

Claim نهایی. حداقل نسخه تنها بعد از تست واقعی ثبت می‌شود.

۷.۲ تغییر تفسیر NFR-۱۷

تصمیم پروژه

«AI» «suggestion» در این پروژه یعنی Recommendation الگوریتمی. شرط زیر ۲ ثانیه برای End-point یا محاسبه پیشنهاد سنجیده می‌شود، نه برای LLM.

۸.۲ منبع WSJF

Epic	WSJF
Setup Account & Onboarding	۸.۰۰
eBooks & Membership Library	۵.۵۷
Experi- Cross-Device Real-Time، ence	۴.۶۰
Multi-Format & Feed Idea Core Content	۳.۸۳
(Text-to- Access Multimodal Speech)	۳.۷۵
Network Direct-Access Creator	۳.۴۰
Awareness Activity & Notifications	۲.۵۰
Discovery Idea AI-Powered	۲.۲۸
Signals Social & Engagement	۲.۲۰

WSJF ترتیب Priority Product را نشان می‌دهد؛ ترتیب Coding باید Dependency-driven باشد. مثلاً Library بدون مدل Idea و User قابل‌پیاده‌سازی معنادار نیست.

۹.۲ تعریف Done برای MVP Core

MVP Core زمانی Done است که Flow واقعی زیر با Database و API واقعی کار کند:

۱. Register/Login؛
۲. Interest؛ + Onboarding
۳. Feed؛
۴. Source؛ + Detail Idea
۵. Spark/Save/Comment؛
۶. Library؛
۷. Follow؛ + Creator
۸. Search/Explore؛
۹. Recommendation؛
۱۰. Persian/English.

فصل ۳

فلسفه توسعه و روش یادگیری

۱.۳ اصل اول: پروژه باید قابل فهم بماند

هر کتابخانه یا Framework هزینه ذهنی دارد. اضافه‌کردن Dependency زمانی مجاز است که:

۱. یک نیاز واقعی را حل کند؛
۲. راه ساده‌تر built-in کافی نباشد؛
۳. Maintenance مناسبی داشته باشد؛
۴. بتوانیم توضیح دهیم چه کاری انجام می‌دهد؛
۵. Test و Debug آن قابل مدیریت باشد.

۲.۳ اصل دوم: Magic Before Learn

چیزی که باید یاد بگیری

قرار نیست Generation Code جای یادگیری را بگیرد. هنگام پیاده‌سازی هر بخش ابتدا Concept توضیح داده می‌شود، سپس مثال کوچک، بعد کد واقعی InsightInk.

مثال:

۱. ابتدا مفهوم Class در Python؛
۲. بعد مفهوم Django Model؛
۳. بعد رابطه Model با Table؛
۴. سپس ساخت Idea؛
۵. سپس Migration؛
۶. سپس Query با ORM؛

۷. سپس Test.

۳.۳ اصل سوم: از دانسته‌های Node.js استفاده می‌کنیم

توسعه‌دهنده با Node.js/Express آشناست. بنابراین برای کاهش بار یادگیری، Concepts جدید با مفاهیم آشنا مقایسه می‌شوند:

Node/Express	Python/Django
npm package	dependency pip / package Python
node_modules	site-packages environment virtual
Express Router	Views + URLConf Django
Middleware	Middleware Django
Prisma/Sequelize Model	Model/ORM Django
Migration tool	Migrations Django
req/res	request/response Django
JWT/session middleware	authentica- Django strategy tion/session/token
validation schema	validation Serializer DRF

۴.۳ اصل چهارم: کوچک‌ترین سیستم درست

در هر فاز ابتدا ساده‌ترین نسخه صحیح ساخته می‌شود.

مثلاً :Recommendation

۱. match؛ Interest
۲. signal؛ Save/Spark
۳. signal؛ Follow
۴. ideas؛ related Jaccard
۵. فقط اگر لازم شد .TF-IDF

نه اینکه از روز اول pipeline ML ساخته شود.

۵.۳ اصل پنجم: Truth of Source Database

تمام داده‌های ماندگار باید در PostgreSQL باشند. WebSocket Redis، cache Browser یا جایگزین Truth of Source نیستند.

۶.۳ اصل ششم: هر Feature شش حالت دارد

برای هر Screen/Feature حداقل این هاState بررسی می‌شوند:

- Idle؛
- Loading؛
- Success؛
- Empty؛
- Error؛
- Offline/Degraded در صورت مرتبط بودن.

۷.۳ اصل هفتم: Done of Definition واقعی

Feature فقط وقتی Done است که:

- رفتار اصلی کار کند؛
- Validation داشته باشد؛
- Permission بررسی شده باشد؛
- state Error وجود داشته باشد؛
- ترجمه فارسی/انگلیسی در نظر گرفته شده باشد؛
- Test مرتبط پاس شود؛
- Documentation در صورت تغییر Contract به‌روز شود.

۸.۳ اصل هشتم: هر Commit یک مفهوم

Commit باید کوچک و قابل مرور باشد. نمونه:

```
feat(content): add Idea and Source models
test(auth): cover moderator permission boundaries
docs(api): document save endpoint
fix(i18n): correct RTL spacing in idea cards
```

۹.۳ اصل نهم: Performance قبل از Optimization کور

تا وقتی اندازه‌گیری نکرده‌ایم، Optimization پیچیده نمی‌کنیم. Index، Redis Cache و Queue بعد از مشاهده نیاز واقعی وارد می‌شوند.

۱۰.۳ اصل دهم: Journal Project

در هر فاز بهتر است یک فایل کوتاه Log نگهداری شود که پاسخ دهد:

- امروز چه چیزی یاد گرفتم؟
- چه چیزی ساختم؟
- چه یBug دیدم؟
- چه یDecision گرفتم؟
- مرحله بعد چیست؟

این Log در دفاع و گزارش نهایی ارزش زیادی دارد.

فصل ۴

معماری کلان سیستم

۱.۴ سبک معماری

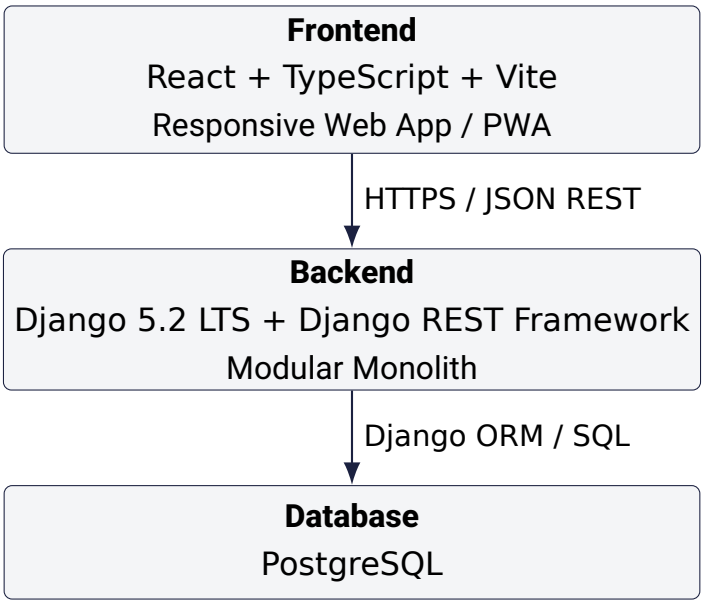
تصمیم پروژه

معماری InsightInk یک Modular Monolith است: یک application Django اصلی، یک Post-greSQL database و ماژول‌های دامنه‌ای جدا در داخل همان Backend.

این انتخاب بین دو افراط قرار می‌گیرد:

- یک پروژه شلوغ که تمام منطق در چند فایل بزرگ مخلوط شده باشد؛
- هایMicroservice زیاد که برای پروژه کارشناسی Complexity بی‌دلیل ایجاد کنند.

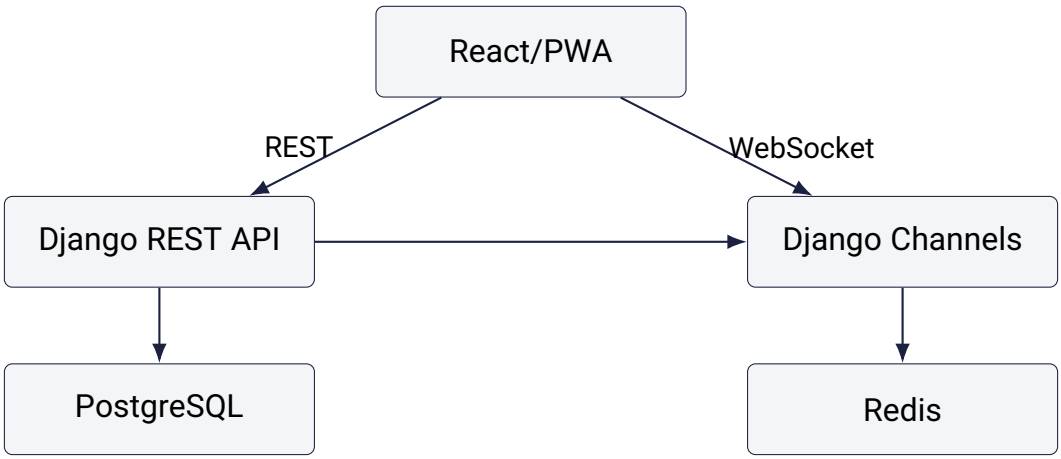
۲.۴ نمودار Baseline



این ساختار برای MVP Core کافی است.

۳.۴ نمودار فاز Realtime

بعداً، فقط هنگام پیاده‌سازی Sync و Messaging:



هشدار / دام رایج

WebSocket منبع حقیقت نیست. ابتدا تغییر باید در PostgreSQL Commit شود؛ سپس Event برای Device دیگر ارسال شود.

۴.۴ مرزهای Backend

ماژول‌های پیشنهادی:

```
backend/
  config/
  apps/
    accounts/
    content/
    creators/
    engagement/
    library/
    recommendations/
    moderation/
    messaging/       # later
    notifications/   # later
```

۱.۴.۴ accounts

مسئول:

- User؛
- settings؛ Profile
- language؛ UI
- selection؛ Interest
- data. domain Authentication-related

۲.۴.۴ content

مسئول:

- Idea؛
- Source؛
- Topic؛
- status؛ Publication
- time. Reading

۳.۴.۴ creators

مسئول:

- CreatorProfile؛
- Follow؛
- data. creator public

۴.۴.۴ engagement

مسئول:

- Spark؛
- Save؛
- Comment.

۵.۴.۴ library

مسئول:

- view؛ ideas Saved
- progress/statistics؛ reading
- data eBook/membership-facing در فاز بعد.

۶.۴.۴ recommendations

مسئول:

- selection؛ Candidate
- scoring؛
- ideas؛ related
- explanation. recommendation

۷.۴.۴ moderation

مسئول:

- Report؛

- ModerationAction؛
- workflow. suspension/hide

۸.۴.۴ messaging و notifications

در Baseline ساخته نمی‌شوند. فقط وقتی فاز مربوط آغاز شد.

۵.۴ چرا Layer Service محدود استفاده می‌کنیم؟

قرار نیست تمام هاViewفقط Wrapper روی هایServiceپیچیده باشند. Layer Service زمانی مفید است که یک Operation: Business

- چند Model را درگیر کند؛
- Transaction بخواهد؛
- از چند Endpoint فراخوانی شود؛
- Rule Business مستقل داشته باشد.

مثال خوب:

- scoring؛ Recommendation
- action؛ moderation
- idea؛ publish
- conversation/message. create

مثال نامناسب:

- یک ()Model.objects.get ساده را فقط برای «تمیز به نظر رسیدن» داخل Service ببریم.

۶.۴ اصل Direction Dependency

ماژول‌ها باید تا حد ممکن از هایDomain پایدارتر به هایDomain سطح بالاتر وابسته باشند.

مثلاً Recommendation می‌تواند از Content و Engagement داده بگیرد؛ ولی Content نباید برای ساخت Idea به Recommendation وابسته باشد.

۷.۴ Search

در ابتدا Search با PostgreSQL پیاده می‌شود:

- title؛
- body؛
- creator؛
- topic؛
- source.

تصمیم پروژه

Elasticsearch/OpenSearch/Meilisearch در Baseline وجود ندارد. فقط اگر Search واقعی پروژه نشان دهد PostgreSQL ناکافی است، در آینده بررسی می‌شود.

۸.۴ Media

avatar Cover، یا فایل eBook در صورت اضافه‌شدن نباید به‌صورت Blob داخل هایModel اصلی ذخیره شوند. Django فقط metadata/path را نگه می‌دارد و Storage File جدا مدیریت می‌شود.

در توسعه محلی می‌توان فایل‌های Media را با Storage محلی Django مدیریت کرد. CDN یا Object Storage فقط در Deployment نهایی و در صورت نیاز واقعی Scope اضافه می‌شود.

۹.۴ اصل Degradation Graceful

اگر Feature اختیاری مانند ،WebSocket Push یا TTS در Browser قدیمی موجود نباشد:

- Login باید کار کند؛
- Feed باید کار کند؛
- Read/Save/Spark/Comment باید کار کند؛
- Library باید کار کند.

Product Core نباید قربانی هایFeature پیشرفته شود.

فصل ۵

Stack Technology و دلیل انتخاب هر ابزار

۱.۵ Backend

۱.۱.۵ Python ۸.۱۴.۳

نسخه موجود روی سیستم توسعه Python 3.14.8 است و با Django نصب‌شده پروژه سازگار است.

کاربرد:

- زبان اصلی Backend؛
- logic؛ Recommendation
- Tests؛
- Scripts و utilities. data

۲.۱.۵ Django ۱۸.۲.۵

Framework Django اصلی Backend است.

چیزهایی که از Django استفاده می‌کنیم:

- configuration؛ Project
- Models/ORM؛
- Migrations؛
- foundation؛ Authentication
- Groups/Permissions؛
- Admin؛
- Middleware؛
- Tests؛

- point entry ASGI برای Realtime later.

چیزی که باید یاد بگیری

Django را مثل Express با چند Library جانبی تصور نکن. Django یک Framework -opinion ated است و بخش زیادی از ساختار پروژه را خودش تعریف می‌کند. هدف این است که Conven- هایtion خود Django را یاد بگیریم، نه اینکه معماری Express را زورکی روی آن سوار کنیم.

۳.۱.۵ Framework REST Django

DRF لایه API است.

کاربرد:

- Serializer؛
- Validation؛
- View/ViewSet؛ APIView/Generic
- Permission؛
- Pagination؛
- integration؛ Authentication
- convention. response HTTP

DRF بعد از فهم Django پایه وارد پروژه می‌شود.

۴.۱.۵ PostgreSQL

Database اصلی و production-like پروژه.

چرا:

- NFR-۱۱ صریحاً DB relational SQL می‌خواهد؛
- User/Idea/Topic/Save/Comment/Follow روابط واضح دارند؛
- Constraint و Index قوی؛
- integration Django عالی؛
- قابلیت Search Text Full و pg_trgm برای future. Search

۵.۱.۵ Redis – فقط بعداً

Redis در Day ۱ نصب نمی‌شود.

وقتی Realtime شروع شد:

- layer؛ Channels
- state؛ realtime transient
- احتمالاً cache محدود.

data Persistent در Redis نگهداری نمی‌شود.

۶.۱.۵ Channels Django – فقط بعداً

برای:

- events؛ sync cross-device
- messaging؛ direct
- arrival. notification realtime

۷.۱.۵ Celery – فعلاً خیر

در Baseline وجود ندارد.

فقط اگر Feature واقعی مثل:

- batching؛ scheduled
- audio؛ TTS generated
- jobs؛ email heavy
- processing background long-running

نیاز واقعی ایجاد کرد، دوباره ارزیابی می‌شود.

۲.۵ Frontend

۱.۲.۵ HTML

قبل از React باید HTML Semantic فهمیده شود:

- heading؛
- nav؛
- main؛
- article؛

- section؛
- form؛
- label/input؛
- button/link.

این موضوع برای Accessibility و Browser قدیمی حیاتی است.

۲.۲.۵ CSS

موضوعات کلیدی:

- Model؛ Box
- Cascade/Specificity؛
- Flexbox؛
- Grid؛
- Design؛ Responsive
- Variables؛ CSS
- Properties؛ Logical
- RTL/LTR؛
- states. Focus/hover/disabled

۳.۲.۵ JavaScript

حتی با سابقه، Concepts JavaScript Browser Node.js باید یاد گرفته شود:

- DOM؛
- Events؛
- Fetch؛
- storage؛ Browser
- History/navigation؛
- async/await در Context شبکه.

۴.۲.۵ TypeScript

Frontend Production با TypeScript نوشته می‌شود.

مزایا:

- type؛ safety
- refactor امن‌تر؛
- contract data API واضح‌تر؛
- autocomplete بهتر؛
- Skill رزومه‌ای مفید.

۵.۲.۵ React

React برای Component-based UI استفاده می‌شود.

هایConcept لازم:

- Component؛
- JSX؛
- Props؛
- State؛
- handling؛ Event
- rendering؛ Conditional
- list/key؛
- hooks؛
- composition.

۶.۲.۵ Vite

Vite UI Framework نیست. ابزار Server Development و Build است.

کاربرد:

- server؛ dev
- bundling؛
- frontend؛ variables environment
- build. production

۷.۲.۵ Router React

برای هاییroute مانند:

```
/login
/onboarding
```

```
/feed
/ideas/:id
/explore
/library
/creators/:id
/profile
```

۸.۲.۵ Query TanStack

وقتی Frontend به API واقعی وصل شد، State Server با TanStack Query مدیریت می‌شود:

- query؛
- cache؛
- loading/error؛
- mutation؛
- invalidation؛
- update؛ optimistic
- pagination. infinite

تصمیم پروژه

Redux و Zustand از ابتدا نصب نمی‌شوند. اگر بعداً State Client واقعی و پیچیده داشتیم، آن زمان ارزیابی می‌کنیم.

۹.۲.۵ next۱۸react-i / next۱۸i

برای ترجمه UI فارسی و انگلیسی.

۳.۵ Stack Testing

۱.۳.۵ Backend

- pytest
- pytest-django

۲.۳.۵ Frontend

- Vitest
- React Testing Library

۳.۳.۵ End-to-End

Playwright بعد از Integration واقعی .Front/Back

۴.۳.۵ Test Load

در فاز ،Performance ابزارهایی مانند Locust برای سنجش هاNFR قابل استفاده‌اند.

۴.۵ Quality Code

Backend:

- Ruff؛
- hints؛ type
- policy. formatter/linter

Frontend:

- ESLint؛
- Prettier؛
- strictness TypeScript در حد عملی.

۵.۵ ابزارهایی که انتخاب نشده‌اند

ابزار	دلیل عدم انتخاب در Baseline
Next.js	framework Server-side دوم کنار ؛Django نیاز فوری ندارد.
Flutter	یادگیری Dart و مسیر جدا؛ از دانش JavaScript موجود استفاده نمی‌کند.

Ionic/Capacitor	برای هدف هایDevice قدیمی، محدودیت Native wrapper می‌تواند مشکل‌ساز شود؛ Web/PWA-first ساده‌تر است.
GraphQL	REST برای این Domain کافی و آموزشی‌تر است.
MongoDB	Domain رابطه‌ای است و SQL NFR می‌خواهد.
Elasticsearch	Search اولیه با PostgreSQL کافی است.
DB Vector	Embedding وجود ندارد.
SDK LLM	LLM در Scope نیست.
Microservices	Complexity غیرضروری.
Kubernetes	خارج از نیاز پروژه کارشناسی.

۶.۵ قاعده انتخاب Dependency

قبل از pip install یا npm install باید بتوانیم پاسخ دهیم:

۱. کدام Requirement به این Package نیاز دارد؟
۲. آیا Library Standard یا Framework فعلی این کار را انجام نمی‌دهد؟
۳. این Dependency چه Complexity جدیدی می‌آورد؟
۴. چطور Test می‌شود؟
۵. اگر حذف شود چه چیزی می‌شکند؟

فصل ۶

ساختار Repository و آماده‌سازی محیط

۱.۶ وضعیت فعلی سیستم

مسیر فعلی :Repository

```
/home/Pandatron/PycharmProjects/InkSight
```

نام :Product

InsightInk

نسخه‌های تأییدشده:

```
Python 3.14.8
Django 5.2.18
```

Repository در حال حاضر Git دارد و venv. نیز در Root موجود است.

۲.۶ ساختار نهایی Monorepo

```
InkSight/├──
 backend/|├──
    config/|├──
    apps/|├──
    manage.py|└──
    ...├──
 frontend/|├──
    src/|├──
    public/|├──
```

```
    package.json└──
    ...├──
docs/├──
    latex/└──
    ...├──
.gitignore├──
README.md└──
...
```

Root فقط Container کل پروژه است. نه Project Django است و نه Project. Vite

۳.۶ Environment Virtual

venv. در Root برای یک پروژه تک‌توسعه‌دهنده قابل قبول است.

وظیفه Environment: Virtual

- جدا کردن هایPackage پروژه از Python سیستم؛
- ؛reproducibility
- جلوگیری از conflict نسخه‌ها.

بررسی:

```
.venv/bin/python --version
.venv/bin/python -m django --version
```

۴.۶ Cleanup Git قبل از کدنویسی

وضعیت فعلی:

- /idea. در commit اولیه Track شده؛
- main.py یک starter PyCharm است؛
- /venv. خوشبختانه Track نشده؛
- Branch فعلی هنگام بررسی master بوده است.

در Phase ۰ باید آگاهانه تصمیم بگیریم:

۱. آیا Repository را از InkSight به InsightInk rename کنیم؟
۲. آیا Branch را به main تغییر دهیم؟
۳. /idea. را از Git خارج کنیم؟

۴. `main.py` را حذف کنیم؟

۵.۶ Gitignore پیشنهادی

```
# Python
.venv/
__pycache__/
*.py[cod]

# Secrets
.env
.env.*
!.env.example

# Django/runtime
*.sqlite3
media/
staticfiles/

# Frontend
node_modules/
dist/
coverage/

# IDE
.idea/

# OS
.DS_Store
Thumbs.db
```

فقط مواردی را Ignore می‌کنیم که واقعاً Generated/Private هستند.

۶.۶ ساخت Backend

وقتی Phase ۱ شروع شد:

```
mkdir backend
cd backend
../.venv/bin/django-admin startproject config .
```

نقطه آخر یعنی Project را داخل Directory Current بساز، نه اینکه یک nested Folder اضافی تولید شود.

خروجی تقریبی:

```
backend/├──
 config/│├──
    __init__.py│├──
    asgi.py│├──
    settings.py│├──
    urls.py│└──
    wsgi.py└──
 manage.py
```

۷.۶ ساخت Frontend

در Phase مربوط:

```
mkdir frontend
cd frontend
npm create vite@latest . -- --template react-ts
```

نسخه و Command دقیق در زمان اجرا دوباره بررسی می‌شود.

۸.۶ PostgreSQL

قبل از هایModel واقعی باید:

۱. PostgreSQL نصب/فعال باشد؛
۲. Database پروژه ساخته شود؛
۳. User/Password محلی مشخص شود؛
۴. connection Django تست شود.

هشدار / دام رایج

برای پروژه واقعی، SQLite را به‌عنوان Database نهایی نگه نمی‌داریم. اگر برای یک تمرین آموزشی کوتاه استفاده شد، خیلی زود به PostgreSQL مهاجرت می‌کنیم.

۹.۶ Variables Environment

بعداً حداقل:

```
DJANGO_SECRET_KEY=
DJANGO_DEBUG=
DATABASE_NAME=
DATABASE_USER=
DATABASE_PASSWORD=
DATABASE_HOST=
DATABASE_PORT=
```

فایل `.env` Commit نمی‌شود؛ `.env.example` بدون Secret می‌تواند Commit شود.

۱۰.۶ Files Dependency

برای Python باید یک روش reproducible انتخاب شود. گزینه‌ها:

- `requirements.txt` ساده؛
- یا `pyproject.toml` با ابزار مدرن.

برای پروژه آموزشی، سادگی مهم است. تصمیم نهایی در Phase ۰ گرفته می‌شود و بعد ثابت می‌ماند.

dependencies Frontend در `package.json` و lockfile مدیریت می‌شوند.

۱۱.۶ روال روزانه توسعه

۱. `git status`
۲. خواندن Task همان Phase
۳. یادگیری Concept
۴. تغییر کوچک
۵. اجرای Test
۶. `git diff`
۷. Documentation Update در صورت تغییر Contract
۸. Commit

۱۲.۶ Done of Definition برای Phase ۰

معیار پایان این مرحله

Phase ۰ وقتی تمام است که:

- structure Root تمیز باشد؛
- Gitignore درست باشد؛
- files IDE/generated مدیریت شده باشند؛
- نسخه Python/Django ثبت شده باشد؛
- PostgreSQL آماده باشد؛
- dependency Strategy مشخص باشد؛
- اولین Commit مستندات ثبت شده باشد.

فصل ۷

مسیر یادگیری Python و Django

۱.۷ هدف این فصل

چون Backend قبلی توسعه‌دهنده Node.js/Express بوده، این فصل ترتیب یادگیری را طوری می‌چیند که دانش قبلی حفظ شود و تفاوت‌های Python/Django به‌صورت مرحله‌ای فهمیده شوند.

۲.۷ Python: چیزهایی که واقعاً لازم داریم

۱.۲.۷ مرحله ۱P: Syntax و Types Data

باید بتوانی این موارد را بدون کمک توضیح بدهی و بنویسی:

- variable؛
- str/int/float/bool/None؛
- list؛
- tuple؛
- dict؛
- set؛
- if/elif/else؛
- for/while.

مثال:

```
topics = ["science", "history", "business"]

for topic in topics:
    print(topic)
```

۲.۲.۷ مرحله ۲P: Function

```
def calculate_reading_minutes(word_count: int) -> int:
    words_per_minute = 220
    return max(1, round(word_count / words_per_minute))
```

مفاهیم:

- parameter؛
- argument؛
- return؛
- parameter؛ default
- hint؛ type
- scope.

۳.۲.۷ مرحله ۳P: Module و Package

در Node.js با import/export آشنا هستی. در Python نیز Code را به هاModule تقسیم می‌کنیم.

```
from recommendations.scoring import calculate_score
```

باید تفاوت این‌ها را بفهمی:

- file/module؛
- package؛
- path؛ import
- `__init__.py`.

۴.۲.۷ مرحله ۴P: Class و Object

Django هاModel Class هستند؛ بنابراین قبل از Model باید OOP پایه را بفهمی.

```
class IdeaSummary:
    def __init__(self, title: str, score: float):
        self.title = title
        self.score = score

    def is_recommended(self) -> bool:
        return self.score >= 5
```

مفاهیم:

- class؛
- instance؛
- attribute؛
- method؛
- constructor؛
- inheritance؛
- composition.

۵.۲.۷ مرحله ۵P: Exception

```
try:
    score = calculate_score(user, idea)
except ValueError as exc:
    logger.warning("Invalid recommendation input: %s", exc)
```

نباید هر Exception را با except Exception پنهان کنیم.

۶.۲.۷ مرحله ۶P: Comprehension و Iterator

در حد عملی:

```
topic_ids = [topic.id for topic in idea.topics.all()]
```

هدف فهم خوانایی و behavior Lazy است، نه استفاده نمایشی از Syntax پیچیده.

۳.۷ Django از چه اجزایی تشکیل شده؟

۱.۳.۷ Project

Django Project تنظیمات کل Application است.

فایل‌های اصلی:

- settings.py
- urls.py
- asgi.py
- wsgi.py

۲.۳.۷ App

App Django یک Domain/Capability قابل تفکیک است.

مثلاً:

```
accounts
content
engagement
recommendations
```

هشدار / دام رایج

برای هر Table یک App نساز. App باید Responsibility Business داشته باشد.

۴.۷ Routing URL

Express:

```
app.get("/ideas/:id", handler)
```

Django:

```
path("ideas/<int:pk>/", views.idea_detail)
```

DRF در مراحل بعد هایabstraction بیشتری می‌دهد، اما ابتدا باید این مسیر ساده فهمیده شود.

۵.۷ Request و Response

باید بفهمی:

- request Browser می‌فرستد؛
- URL Django را Resolve می‌کند؛
- View اجرا می‌شود؛
- query DB در صورت نیاز انجام می‌شود؛
- Response برمی‌گردد.

۶.۷ Model و ORM

مثال آموزشی:

```python
from django.db import models

class Topic(models.Model):
    slug = models.SlugField(unique=True)
    name_fa = models.CharField(max_length=100)
    name_en = models.CharField(max_length=100)
```

این Class به Table در Database نگاشت می‌شود.

باید این عملیات ORM را بلد باشی:

```python
Topic.objects.all()
Topic.objects.get(slug="science")
Topic.objects.filter(is_active=True)
Topic.objects.create(...)
```

۷.۷ Migration

Migration تاریخچه تغییر Schema است.

Flow:

```
change models.py↓

python manage.py makemigrations↓

inspect migration↓

python manage.py migrate
```

هشدار / دام رایج

Migration را فایل Generated بی‌ارزش تصور نکن. بخشی از تاریخچه Schema پروژه است و باید Commit شود.

۸.۷ Admin Django

Admin برای پروژه ما بسیار ارزشمند است چون قبل از ساخت CMS Creator می‌توانیم:

- Topic بسازیم؛

- Creator ایجاد کنیم؛
- Source وارد کنیم؛
- publish Idea کنیم؛
- هاReport را بررسی کنیم.

بنابراین Data واقعی خیلی زود وارد Backend می‌شود.

۹.۷ User Custom از روز اول

هشدار / دام رایج

تغییر AUTH_USER_MODEL بعد از هایMigration متعدد دردسر زیادی ایجاد می‌کند. بنابراین قبل از اولین Migration واقعی، User Custom را می‌سازیم.

حتی اگر User Custom اولیه ساده باشد، انعطاف آینده را حفظ می‌کند.

۱۰.۷ Authentication Django

باید مفاهیم زیر را بفهمی:

- password؛ hashing
- authenticate؛
- login/logout؛
- session؛
- user؛ authenticated
- group؛
- permission؛
- superuser.

۱۱.۷ DRF بعد از Django پایه

بعد از اینکه Model/View/URL را فهمیدی، می‌رویم سراغ:

- Serializer؛
- validation؛
- APIView؛

- views؛ generic
- ViewSet؛
- Router؛
- Permission؛
- Pagination.

۱۲.۷ ترتیب تمرین‌های Backend

۱. یک endpoint ساده health؛ check
۲. model Topic + admin؛
۳. model؛ Source
۴. model؛ Idea
۵. query با shell؛
۶. read-only؛ API Topic
۷. read-only؛ API Idea
۸. validation؛ create/update
۹. auth؛
۱۰. permissions؛
۱۱. endpoints. engagement

۱۳.۷ معیار فهم

قبل از رفتن به مرحله بعد باید بتوانی پاسخ بدهی:

- Model چه فرقی با Serializer دارد؟
- Migration دقیقاً چه چیزی را تغییر می‌دهد؟
- QuerySet چیست؟
- چرا ForeignKey با IntegerField متفاوت است؟
- URLConf چه نقشی دارد؟
- Request چطور به View می‌رسد؟
- Permission کجا اعمال می‌شود؟

فصل ۸

طراحی Backend و ساختار Django

۱.۸ ساختار پیشنهادی

```
backend/├──
 manage.py├──
 config/|├──
    settings.py|├──
    urls.py|├──
    asgi.py|└──
    wsgi.py├──
 apps/|├──
    accounts/|├──
    content/|├──
    creators/|├──
    engagement/|├──
    library/|├──
    recommendations/|├──
    moderation/|├──
    messaging/|└──
    notifications/└──
 tests/
```

در عمل ممکن است هایTest هر App داخل همان App باشند. هدف این Diagram نشان دادن ،هاستDomain نه اجبار به یک Layout خاص.

۲.۸ config

config/ فقط Configuration پروژه است:

- settings؛
- URLs: root؛
- ASGI/WSGI؛
- later. configuration environment-specific

Model Business داخل config قرار نمی‌گیرد.

۳.۸ accounts

هایModel احتمالی:

- User؛
- UserInterest؛
- UserPreference در صورت نیاز.

هایService احتمالی:

- interests: update؛
- later. export/delete data account

۴.۸ content

هسته :Domain

- Topic؛
- Source؛
- Idea؛
- IdeaTopic در صورت نیاز به Model. Through

قواعد:

- فقط Idea Published در Feed عمومی؛
- content Hidden در Feed ظاهر نمی‌شود؛
- هر Idea باید Creator و Source مشخص داشته باشد؛
- Language باید مشخص باشد؛
- time reading estimated یا محاسبه شود یا قبل از publish ثبت شود.

۵.۸ creators

- CreatorProfile؛
- CreatorFollow.

CreatorProfile با OneToOne به User وصل می‌شود.

۶.۸ engagement

- Spark؛
- SavedIdea؛
- Comment.

Spark و Save باید Constraint Unique روی (user, idea) داشته باشند.

۷.۸ library

در فاز اولیه Library می‌تواند صرفاً Query روی SavedIdea باشد.

تصمیم پروژه

Table جدا برای LibraryEntry فقط زمانی ساخته می‌شود که یRequirement وجود داشته باشد که Save و Item Library را از هم متمایز کند.

بعداً:

- stats؛ reading
- eBooks؛
- entitlements. membership

۸.۸ recommendations

Model Recommendation اجباری نیست.

در ابتدا:

- Query هاcandidate؛
- score در Service؛
- response.

اگر بعداً caching لازم شد، data derived طراحی می‌شود.

۹.۸ moderation

- Report؛
- ModerationAction.

این App باید boundary Permission واضح داشته باشد.

۱۰.۸ Logic Business کجا قرار بگیرد؟

قاعده ساده:

۱.۱۰.۸ Model

قواعد محلی و invariants data نزدیک خود .Entity

۲.۱۰.۸ Serializer

Input/output transformation و API-level. validation

۳.۱۰.۸ View

response. logic، business call permission، request، orchestration: HTTP

۴.۱۰.۸ Service

operation Business چندمرحله‌ای یا قابل‌استفاده مجدد.

۵.۱۰.۸ helper Selector/Query

هایQuery پیچیده .read-side

۱۱.۸ Transaction

هایی‌Operation که چند write وابسته دارند باید Transaction داشته باشند.

مثال :future

- state؛ target change + action moderation create
- entitlement. + activation membership

۱۲.۸ Signals

Signals Django جذاب‌اند اما باید محدود استفاده شوند.

هشدار / دام رایج

Logic Business مهم را پشت های‌Signal نامرئی پنهان نکن. اگر یک Action بخشی از Flow اصلی است، call service explicit معمولاً قابل‌فهم‌تر است.

Signals برای های‌side-effect محدود و کاملاً مستند مناسب‌اند.

۱۳.۸ Settings

ابتدا یک `settings.py` ساده کافی است. اگر بعداً Production/Development تفاوت زیاد شد می‌توان split کرد.

اصل:

- Secret از ؛env
- DEBUG در production ؛false
- config Database از ؛env
- hosts Allowed مشخص؛
- CORS/CSRF بر اساس deployment واقعی.

۱۴.۸ Logging

از ()print برای Debug موقت می‌توان استفاده کرد، اما Logging واقعی باید از logging/Django Python استفاده کند.

سطوح:

- DEBUG؛
- INFO؛
- WARNING؛
- ERROR.

اطلاعات حساس نباید log شود.

۱۵.۸ strategy Admin

برای هر Model لازم نیست Admin بسیار پیچیده بسازیم.

ولی برای:

- Idea؛
- Source؛
- Topic؛
- CreatorProfile؛
- Report

List ،search display و filters مفید هستند.

۱۶.۸ data Seed

برای توسعه، Dataset کوچک و deterministic می‌سازیم:

- چند User؛
- چند Creator؛
- ۱۰-۲۰ Topic؛
- چند Source از هر نوع؛
- حداقل ۳۰-۵۰ Idea برای test. Feed/Recommendation

این داده‌ها باید یا Command Fixture/Management قابل تکرار باشند، نه ورود دستی غیرقابل بازسازی.

۱۷.۸ Command Management

یک Command مثل:

```
python manage.py seed_demo
```

می‌تواند data Demo بسازد.

این بخش بعد از فهم basics اضافه می‌شود.

۱۸.۸ Done of Definition Backend

هر feature: Backend

- Model/constraint درست؛
- Migration؛
- validation؛
- permission؛
- tests؛
- docs؛ API
- setup admin/data در صورت لازم.

فصل ۹

طراحی Database و مدل داده

۱.۹ اصل طراحی

۱۱NFR- استفاده از SQL Relational را مشخص می‌کند. Domain نیز به‌شدت Relation-oriented است؛ بنابراین PostgreSQL انتخاب طبیعی است.

۲.۹ User

```
User
- id
- email
- password_hash (managed by Django)
- display_name
- preferred_ui_language
- is_active
- is_staff
- date_joined
- created_at / updated_at if needed
```

تصمیم پروژه

Model User Custom قبل از اولین Migration واقعی ساخته می‌شود.

۳.۹ CreatorProfile

```
CreatorProfile
```

```
- id
- user_id -> User (OneToOne)
- bio_fa
- bio_en
- avatar
- is_verified
- created_at
```

Topic ۴.۹

```
Topic
- id
- slug
- name_fa
- name_en
- is_active
```

Slug باید unique باشد.

UserInterest ۵.۹

```
UserInterest
- user_id
- topic_id
- created_at
```

Constraint:

```
UNIQUE(user_id, topic_id)
```

Source ۶.۹

Source کار اصلی است که Idea از آن آمده:

```
Source
- id
- type: BOOK | ARTICLE | MOVIE | PODCAST
- title
```

```
- original_creator_text or structured relation if later needed
- language
- external_url (optional)
- cover (optional)
- metadata (limited)
```

هشدار / دام رایج

Creator Source با Creator InsightInk لزوماً یک چیز نیست. مثلاً یک کاربر InsightInk ممکن است درباره کتاب یک نویسنده خارجی Idea بنویسد.

۷.۹ Idea

```
Idea
- id
- creator_id -> CreatorProfile
- source_id -> Source
- language
- title
- body
- estimated_read_seconds
- status: DRAFT | PUBLISHED | HIDDEN
- published_at
- created_at
- updated_at
```

Relation:

$$Idea \leftrightarrow Topic$$

۸.۹ Spark

```
Spark
- user_id
- idea_id
- created_at

UNIQUE(user_id, idea_id)
```

۹.۹ SavedIdea

```
SavedIdea
- user_id
- idea_id
- saved_at
- last_opened_at (optional)

UNIQUE(user_id, idea_id)
```

۱۰.۹ Comment

```
Comment
- id
- author_id -> User
- idea_id -> Idea
- body
- status: VISIBLE | HIDDEN | DELETED
- created_at
- updated_at
```

Comment Nested در requirement Source ضروری نیست و فعلاً اضافه نمی‌شود.

۱۱.۹ CreatorFollow

```
CreatorFollow
- user_id
- creator_id
- created_at

UNIQUE(user_id, creator_id)
```

۱۲.۹ ReadingEvent

برای Story «چند Idea این ماه خواندم» باید اول تعریف Read را مشخص کنیم.

Model احتمالی:

```
ReadingEvent
- id
- user_id
- idea_id
- occurred_at
```

هشدار / دام رایج

قبل از تعریف دقیق «read»، این Table را نساز. صرف باز شدن صفحه ممکن است معادل خواندن واقعی نباشد.

۱۳.۹ Report

```
Report
- id
- reporter_id
- target_type
- target_id
- reason
- details
- status
- created_at
- resolved_at
```

برای target Generic باید بین GenericForeignKey و هایTable مشخص تصمیم بگیریم. برای پروژه آموزشی، سادگی و Queryability مهم‌تر از abstraction Generic است.

۱۴.۹ ModerationAction

```
ModerationAction
- id
- moderator_id
- report_id (optional)
- action_type
- target information
- note
- created_at
```

۱۵.۹ later – Messaging

```
Conversation
- id
- created_at

ConversationParticipant
- conversation_id
- user_id

Message
- id
- conversation_id
- sender_id
- body
- created_at
- read_at
```

برای Scope اولیه Conversation بین Reader و Creator است.

۱۶.۹ later – Notification

```
Notification
- id
- recipient_id
- type
- payload
- read_at
- created_at

NotificationPreference
- user_id
- notification_type
- enabled
```

۱۷.۹ later – Membership/eBook

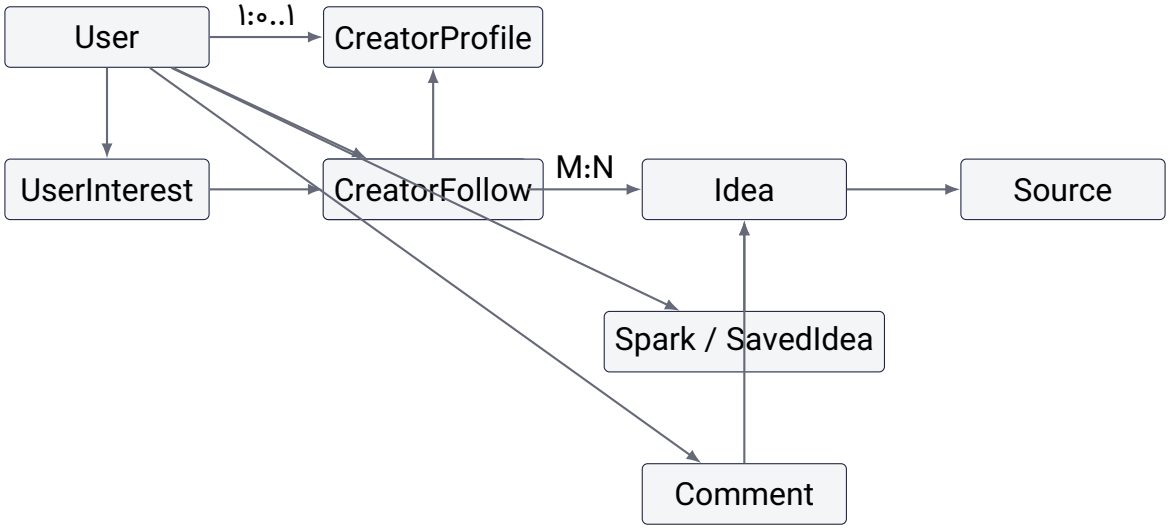
```
Ebook
- id
```

```
- source_id
- title
- language
- file_metadata
- access_level

Membership
- id
- user_id
- provider_reference
- status
- plan
- period_start
- period_end
```

data card Raw جایی در Schema ما ندارد.

۱۸.۹ ERD مفهومی



۱۹.۹ Strategy Index

Index روی هاییField که مرتب Filter/Sort می‌شوند:

- Idea.status؛
- Idea.published_at؛
- Idea.language؛

- Topic.slug؛
- Source.type؛
- later. state recipient/read Notification

join Many-to-many هاtable و Foreign هاKey معمولاً Index مناسب دارند، ولی باید plan Query را بعداً بررسی کنیم.

۲۰.۹ Normalization

اصل:

- داده تکراری را بی‌دلیل Duplicate نکن؛
- Relation واقعی را ForeignKey کن؛
- JSONField فقط برای metadata واقعاً ؛flexible
- Denormalization فقط بعد از performance. Evidence

۲۱.۹ Rules Migration

۱. Model را تغییر بده؛
۲. `makemigrations`؛
۳. فایل Migration را بخوان؛
۴. `migrate`؛
۵. Test؛
۶. Migration + Model Commit با هم.

فصل ۱۰

REST، Authentication API و هاRole

۱.۱۰ Base API

تمام هایEndpoint: Versioned

```
/api/v1/
```

فرمت اصلی:

```
Content-Type: application/json
```

۲.۱۰ هاMethod HTTP

Method	کاربرد
GET	خواندن Resource
POST	ساخت Resource یا non-idempotent Action
PUT	جایگزینی/idempotent Action در طراحی مناسب
PATCH	تغییر بخشی از Resource
DELETE	حذف Resource یا Relationship

۳.۱۰ Core Endpointهای پیشنهادی

۱.۳.۱۰ Account

```
POST   /api/v1/auth/register/
POST   /api/v1/auth/login/
POST   /api/v1/auth/logout/

GET    /api/v1/me/
PATCH  /api/v1/me/

GET    /api/v1/me/interests/
PUT    /api/v1/me/interests/
```

۲.۳.۱۰ Feed/Idea

```
GET    /api/v1/feed/
GET    /api/v1/ideas/{id}/
GET    /api/v1/ideas/{id}/related/
```

۳.۳.۱۰ Engagement

```
PUT    /api/v1/ideas/{id}/spark/
DELETE /api/v1/ideas/{id}/spark/

PUT    /api/v1/ideas/{id}/save/
DELETE /api/v1/ideas/{id}/save/

GET    /api/v1/ideas/{id}/comments/
POST   /api/v1/ideas/{id}/comments/
DELETE /api/v1/comments/{id}/
POST   /api/v1/comments/{id}/report/
```

Spark/Save باید idempotent طراحی شوند؛ یعنی تکرار duplicate PUT نسازد.

۴.۳.۱۰ Library

```
GET    /api/v1/library/saved/
GET    /api/v1/library/stats/
```

۵.۳.۱۰ Creator

```
GET    /api/v1/creators/
GET    /api/v1/creators/{id}/
PUT    /api/v1/creators/{id}/follow/
DELETE /api/v1/creators/{id}/follow/
```

۶.۳.۱۰ Search

```
GET    /api/v1/search/?q=...
GET    /api/v1/topics/
GET    /api/v1/topics/{slug}/ideas/
```

۷.۳.۱۰ Recommendation

```
GET    /api/v1/recommendations/
GET    /api/v1/ideas/{id}/related/
```

هیچ endpoint Completion/Chat وجود ندارد.

۴.۱۰ Pagination

Feed برای content long-running بهتر است pagination Cursor-like داشته باشد تا Insert شدن محتوای جدید duplicate/missing کمتری ایجاد کند.

Response مفهومی:

```
{
  "next": "opaque-cursor",
  "previous": null,
  "results": [...]
}
```

Implementation دقیق با DRF در زمان فاز Feed انتخاب می‌شود.

۵.۱۰ Authentication

Authentication پاسخ می‌دهد:

«این Request متعلق به چه کسی است؟»

نیازها:

- email/password؛
- hash؛ password secure
- login/logout؛
- reset؛ password
- request؛ API authenticated
- login روی Device دیگر.

Login Social در Source آمده اما هنوز Scope نهایی آن Question Open است.

۶.۱۰ Token؟ یا Session

این تصمیم را قبل از دانستن topology Deployment قفل نمی‌کنیم.

اگر Frontend/API same-origin باشند، approach Cookie/Session بسیار طبیعی است. اگر Origins جدا یا نیازهای PWA خاص داشته باشیم، strategy Token بررسی می‌شود.

هشدار / دام رایج

فقط چون JWT معروف است، system JWT Custom نمی‌سازیم. ابتدا model Threat و De-ployment را می‌فهمیم.

۷.۱۰ Authorization

Authorization پاسخ می‌دهد:

«این کاربر اجازه انجام این Action روی این Object را دارد؟»

۸.۱۰ Reader

های‌Permission اصلی:

- content؛ public read
- save/spark؛
- comment؛
- follow؛
- report؛
- profile/interests؛ own
- later. creator message

۹.۱۰ Creator

همه قابلیت‌های Reader +:

- profile؛ creator
- publishing/editing idea own در صورت ساخت ؛UI/API
- statistics؛ own
- reply. message

ownership Object حیاتی است.

۱۰.۱۰ Moderator

- reports؛ view
- content؛ flagged review
- hide/remove طبق ؛policy
- report؛ resolve
- محدودیت‌های تعریف‌شده.

نمی‌تواند:

- Moderator بسازد؛
- Admin بسازد؛
- هر تنظیم سیستمی را تغییر دهد.

۱۱.۱۰ Admin

دسترسی کامل. در Django معمولاً با Superuser و هایpermission کامل نگاشت می‌شود.

۱۲.۱۰ Strategy Group/Permission

نگاشت پیشنهادی:

```
Reader    -> authenticated user
Creator   -> User + CreatorProfile + creator permissions
Moderator -> Moderator Group (+ limited staff/admin access)
Admin     -> superuser
```

۱۳.۱۰ Example Permission Object-Level

سناریو:

- A Idea A Creator را دارد؛
- B Creator لاگین است.

B Creator نباید بتواند:

```
PATCH /api/v1/ideas/A/
```

حتی اگر Role او Creator باشد.

۱۴.۱۰ Format Error Validation

response Error باید قابل پیش‌بینی باشد. مثال مفهومی:

```
{
  "errors": {
    "body": ["This field may not be blank."]
  }
}
```

فرمت نهایی هنگام setup DRF مشخص می‌شود.

۱۵.۱۰ OpenAPI

بعد از شکل‌گرفتن API:

- OpenAPI Schema تولید می‌شود؛

- هاEndpoint و Request/Response ثبت می‌شوند؛
- در صورت ارزش عملی، هایFrontend Type از Schema تولید می‌شوند.

Done of Definition API ۱۶.۱۰

هر Endpoint:

- method/path مشخص؛
- requirement؛ auth
- permission؛
- validation؛
- status؛ success/error
- test؛
- documentation.

فصل ۱۱

مسیر یادگیری Frontend از صفر

۱.۱۱ چرا مستقیم وارد React نمی‌شویم؟

React روی مفاهیم Browser و JavaScript بنا شده است. اگر DOM، Fetch Event و State را نفهمیم، React تبدیل به مجموعه‌ای از دستورهای حفظی می‌شود.

بنابراین یادگیری Frontend در این پروژه مرحله‌ای است.

۲.۱۱ مرحله ۱F: HTML

موضوعات:

- structure؛ document
- elements؛ semantic
- hierarchy؛ heading
- button؛ vs link
- form؛
- label/input؛
- nav/main/article/section؛
- basics. accessibility

تمرین پروژه:

- Card Idea استاتیک؛
- form؛ Login
- Navigation ساده.

چیزی که باید یاد بگیری

اول باید بفهمی چرا button برای Action و a برای Navigation استفاده می‌شود. این تفاوت بعداً روی Accessibility و behavior Keyboard تأثیر دارد.

۳.۱۱ مرحله ۲F: CSS

موضوعات:

- selector؛
- cascade؛
- specificity؛
- box model؛
- width/height؛
- margin/padding؛
- Flexbox؛
- Grid؛
- responsive؛
- media query؛
- CSS variable؛
- logical properties؛
- RTL/LTR.

تمرین:

- بازسازی IdeaCard پروتوتایپ؛
- ساخت Bottom Navigation؛
- responsive کردن Feed.

۴.۱۱ مرحله ۳F: JavaScript Browser

با وجود تجربه Node.js، باید concepts Browser-specific را یاد بگیری.

۱.۴.۱۱ DOM

```
const button = document.querySelector("#save-button");
```

```
button.addEventListener("click", () => {
  console.log("saved");
});
```

موضوعات:

- tree؛ DOM
- querySelector؛
- event؛
- target؛ event
- submit؛ form
- preventDefault.

۲.۴.۱۱ Fetch

```
const response = await fetch("/api/v1/ideas/");
const data = await response.json();
```

باید بفهمی:

- Promise؛
- async/await؛
- failure؛ network
- error HTTP با exception جاوااسکریپت یکی نیست؛
- parsing. JSON

۵.۱۱ مرحله ۴F: npm و package.json

مفاهیم:

- package؛
- dependency؛
- devDependency؛
- script؛
- lockfile؛
- versioning semantic در حد عملی.

۶.۱۱ مرحله ۵F: TypeScript

مثال :JavaScript

```
function formatIdea(idea) {
  return idea.title;
}
```

TypeScript:

```
type Idea = {
  id: number;
  title: string;
  body: string;
};

function formatIdea(idea: Idea): string {
  return idea.title;
}
```

موضوعات:

- types؛ primitive
- array/object؛
- alias؛ type
- interface؛
- union؛
- field؛ optional
- type؛ function
- generic در حد لازم؛
- narrowing؛
- any. vs unknown

هشدار / دام رایج

برای سریع شدن، any را به Default تبدیل نکن. اگر داده API را نمی‌دانیم، Contract را تعریف می‌کنیم.

۷.۱۱ مرحله ۶F: Fundamentals React

۱.۷.۱۱ Component

```
function IdeaCard() {
  return <article>...</article>;
}
```

۲.۷.۱۱ Props

```
type IdeaCardProps = {
  title: string;
  readingTime: string;
};

function IdeaCard({ title, readingTime }: IdeaCardProps) {
  return (
    <article>
      <h2>{title}</h2>
      <span>{readingTime}</span>
    </article>
  );
}
```

۳.۷.۱۱ State

مثلاً state local برای باز/بسته بودن Menu.

۴.۷.۱۱ Event

submit. Form switch، Language click، Save

۵.۷.۱۱ Rendering List

Feed:

```
{ideas.map((idea) => (
  <IdeaCard key={idea.id} idea={idea} />
```

```
))}
```

۸.۱۱ مرحله ۷F: Hooks

به ترتیب:

- useState
- useEffect
- useMemo فقط وقتی دلیل وجود دارد
- hook custom
- context.

هشدار / دام رایج

useEffect محل انجام هر نوع Logic نیست. قبل از استفاده باید بفهمیم آیا واقعاً Effect Side داریم یا می‌توان مقدار را مستقیم derive کرد.

۹.۱۱ مرحله ۸F: Routing

هایRoute اصلی:

```
/login
/register
/onboarding
/feed
/ideas/:ideaId
/explore
/library
/creators/:creatorId
/notifications
/profile
```

later: Messaging

```
/messages
/messages/:conversationId
```

۱۰.۱۱ مرحله ۹F: State Server

بعد از اینکه fetch دستی را فهمیدی، Query TanStack وارد می‌شود.

مفاهیم:

- queryKey؛
- queryFn؛
- stale/fresh؛
- cache؛
- mutation؛
- invalidation؛
- update؛ optimistic
- query. infinite

۱۱.۱۱ تمرین‌های Frontend در ترتیب واقعی

۱. IdeaCard Static با HTML/CSS؛
۲. feed؛ mobile Responsive
۳. toggle؛ direction language
۴. fetch ساده از endpoint آزمایشی؛
۵. type TypeScript برای Idea؛
۶. IdeaCard؛ React
۷. feed React از mock array؛
۸. Routing؛
۹. API؛ real fetch
۱۰. Query؛ TanStack
۱۱. mutation؛ Save
۱۲. UI. optimistic

۱۲.۱۱ معیار فهم Frontend

باید بتوانی توضیح دهی:

- DOM چیست؟
- React چه مشکلی را نسبت به manipulation DOM دستی حل می‌کند؟

- Props و State چه فرقی دارند؟
- چرا State Server را با Query TanStack نگه می‌داریم؟
- TypeScript در Runtime وجود دارد یا Compile-time؟
- Vite چه کاری می‌کند و چه کاری نمی‌کند؟

فصل ۱۲

طراحی Frontend و تبدیل Prototype به محصول واقعی

۱.۱۲ Prototype چه چیزی به ما می‌دهد؟

پروتوتایپ موجود ۱۱ Screen دارد و Language Design مشخصی ارائه می‌کند:

- Splash/Brand؛
- Interests؛
- Feed؛ Home
- Detail؛ Idea
- concept؛ exploration AI/Idea
- Explore؛
- Library؛
- Profile؛ Creator
- Messaging؛
- Notifications؛
- Profile/Settings.

تصمیم پروژه

Prototype مرجع UI/UX است، نه code. Production آن به HTML/CSS/JavaScript Inline واقعی تبدیل می‌شود. architecture Component

۲.۱۲ Design هایToken قابل انتقال

Prototype رنگ‌های اصلی را مشخص کرده:

```
--ink:        #12162B
--ink-soft:   #1B2140
--parchment:  #F5F1E6
--spark:      #F2A93B
--tide:       #3E8C8C
--ember:      #E1573A
--ash:        #9497A6
```

فونت‌های :Prototype

- Fraunces؛
- Manrope؛
- Mono. Plex IBM

برای UI فارسی باید خوانایی Vazirmatn یا فونت فارسی مناسب را در frontend Product ارزیابی کنیم؛ لازم نیست فونت Prototype انگلیسی برای متن فارسی استفاده شود.

۳.۱۲ Structure Theme

```
frontend/src/theme/├──
 tokens.css├──
 typography.css├──
 globals.css└──
 components.css   # only if useful
```

۴.۱۲ هایComponent اصلی

۱.۴.۱۲ Primitive/Shared

- Button؛
- IconButton؛
- Badge؛
- Avatar؛
- LoadingState؛

- ErrorState؛
- EmptyState.

۲.۴.۱۲ Components Domain

- IdeaCard؛
- FormatBadge؛
- ReadingTime؛
- SparkButton؛
- SaveButton؛
- CommentList؛
- CreatorSummary؛
- TopicChip؛
- NotificationRow؛
- later؛ MessageBubble
- later. AudioControls

۵.۱۲ folders Feature-based

```
src/├──
 app/├──
 api/├──
 components/├──
 features/|├──
    auth/|├──
    onboarding/|├──
    feed/|├──
    idea/|├──
    explore/|├──
    library/|├──
    creators/|├──
    recommendations/|├──
    messaging/|├──
    notifications/|└──
    profile/├──
 i18n/├──
 theme/├──
 hooks/└──
```

```
types/
```

۶.۱۲ Component vs Page

Page معمولاً:

- Route-level است؛
- fetching data را orchestrate می‌کند؛
- چند Component را کنار هم می‌چیند.

Component:

- بخشی از UI؛
- ترجیحاً مسئولیت محدود؛
- reusable در صورت نیاز واقعی.

۷.۱۲ Design Feed

Feed باید:

- Pagination داشته باشد؛
- page next loading واضح باشد؛
- key duplicate نداشته باشد؛
- performance scroll قابل قبول بماند؛
- روی Device ضعیف Animation سنگین نداشته باشد.

در Frontend:

- Infinite Query در TanStack Query؛
- prefetch نزدیک انتهای لیست در صورت نیاز؛
- skeleton ساده، نه animation سنگین.

۸.۱۲ Contract Data IdeaCard

نمونه Type مفهومی:

```
type IdeaCardData = {
  id: number;
```

```
  title: string;
  excerpt: string;
  language: "fa" | "en";
  source: {
    title: string;
    type: "book" | "article" | "movie" | "podcast";
  };
  creator: {
    id: number;
    displayName: string;
  };
  estimatedReadSeconds: number;
  sparkCount: number;
  commentCount: number;
  isSparked: boolean;
  isSaved: boolean;
};
```

این Type باید از Contract API واقعی derive شود، نه حدس مستقل Frontend.

۹.۱۲ UI Optimistic

برای Save/Spark:

۱. User کلیک می‌کند؛
۲. UI فوراً state را تغییر می‌دهد؛
۳. Request ارسال می‌شود؛
۴. اگر success بود state تثبیت می‌شود؛
۵. اگر failure بود rollback + error message.

این کار به NFR-۰۱ کمک می‌کند.

۱۰.۱۲ Forms

برای Login/Register/Comment:

- label واقعی؛
- validation message؛
- state disabled هنگام submit؛

- error server نمایش داده شود؛
- input؛ keyboard/mobile-friendly
- فارسی/انگلیسی.

در ابتدا می‌توان با state React ساده Form را فهمید؛ بعد اگر Complexity بالا رفت React Hook Form ارزیابی می‌شود. آن را از روز اول اجباری نمی‌کنیم.

۱۱.۱۲ Client API

یک wrapper کوچک برای:

- URL؛ base
- parsing؛ JSON
- credentials/token؛ auth
- handling. error common

هشدار / دام رایج

یک abstraction API غول‌آسا که تمام هاEndpoint و management State را مخلوط کند نساز.

۱۲.۱۲ Strategy Responsive

Mobile-first:

- baseline؛ screen small
- enhancement؛ tablet
- enhancement. layout desktop

نه اینکه Desktop را بسازیم و بعد با ده‌ها query media روی موبایل جمع کنیم.

۱۳.۱۲ Rules UI Device Old

- DOM ساده؛
- shadow/blur/animation محدود؛
- size image کنترل‌شده؛
- media؛ load lazy
- libraries؛ icon/UI huge avoid

- bundle JS کوچک؛
- autoplay. no

۱۴.۱۲ Mapping Screen Prototype

Screen Prototype	Feature Production
Splash	entry auth + intro onboarding
Interests Choose	UI + API interests onboarding
Feed Home	feed personalized paginated
Detail Idea	idea/source/comments/actions
Assistant AI	panel ideas related/recommended non-LLM
Explore	search/topics/formats
Library	stats + ideas saved
Profile Creator	creator/follow/ideas
Creator Message	messaging realtime later
Notifications	center notification in-app later
Profile	language/interests/settings/logout

۱۵.۱۲ Done of Definition Frontend

هر Screen:

- responsive؛
- RTL/LTR؛
- loading؛
- empty؛
- error؛
- name/focus؛ accessible
- contract؛ API
- test component در صورت behavior. critical

فصل ۱۳

سیستم پیشنهاددهی الگوریتمی

۱.۱۳ هدف

Recommendation باید بدون LLM بتواند:

- Feed اولیه را بر اساس Interest بسازد؛
- با رفتار Save/Spark تطبیق پیدا کند؛
- follow Creator را لحاظ کند؛
- محتواهای مشابه را پیدا کند؛
- discovery Cross-format بدهد؛
- دلیل پیشنهاد را شفاف کند.

۲.۱۳ Start Cold

User جدید Behavior ندارد. پس Signal اول:

Interests Declared

مثال:

```
User:
  Science
  Business
  History
```

های‌Idea Score Science/Business/History اولیه بالاتر می‌گیرند.

۳.۱۳ های‌Signal اصلی

- match؛ interest explicit
- affinity؛ saved-topic
- affinity؛ sparked-topic
- creator؛ followed
- popularity؛
- freshness؛
- preference؛ content-language
- penalty. already-seen

۴.۱۳ فرمول پایه

$$Score(i,u) = w_۱I + w_۲S + w_۳K + w_۴F + w_۵P + w_۶R + w_۷L - w_۸V$$

که در آن:

- I: match؛ Interest
- S: affinity؛ Save
- K: affinity؛ Spark
- F: signal؛ creator Followed
- P: Popularity؛
- R: Recency/Freshness؛
- L: fit؛ Language
- V: penalty. Viewed/consumed

وزن اولیه پیشنهادی:

Signal	Weight فرضی
interest Explicit	۵+
affinity topic Saved	۴+
affinity topic Sparked	۳+
creator Followed	۳+
language content Preferred	۲+

Popularity	۰ تا ۱+
Freshness	۰ تا ۱+
consumed Already	penalty

هشدار / دام رایج

این هاRequirement Weight منبع نیستند. Hypothesis فنی هستند و باید با داده آزمایشی Tune شوند.

۵.۱۳ جلوگیری از رشد بی‌نهایت Affinity

اگر User صدها Save روی یک Topic داشت، Score نباید خطی بی‌نهایت رشد کند.

مثال ساده:

$$Affinity = \min(count, cap)$$

یا Normalization به بازه مشخص.

۶.۱۳ Similarity Jaccard

برای Ideas: Related

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

مثال:

$$A = \{Productivity, Focus, Psychology\}$$
$$B = \{Productivity, Focus, Habits\}$$
$$J(A,B) = \frac{۲}{۴} = ۰/۵$$

مزیت:

- deterministic؛
- قابل توضیح؛
- بدون Model/Embedding؛
- مناسب دفاع دانشگاهی.

Diversity Cross-format ۷.۱۳

اگر Idea User کتابی درباره Focus می‌بیند، list Related می‌تواند Podcast با Topic مشابه را Boost کند.

مفهوم:

$$FinalRelatedScore = TopicSimilarity + DiversityBonus$$

Format متفاوت نباید خودکار بهتر باشد؛ فقط Bonus کوچک برای جلوگیری از monotony.

Popularity ۸.۱۳

Popularity نباید صرفاً sparks total باشد چون محتوای قدیمی همیشه برنده می‌شود.

می‌توانیم از popularity Normalized استفاده کنیم:

- sparks در بازه اخیر؛
- comments؛
- save ratio؛
- capped score.

برای MVP ساده نگه می‌داریم.

Freshness ۹.۱۳

Freshness می‌تواند تابعی از age باشد:

$$Freshness = \max(۰, ۱ - \frac{AgeDays}{Window})$$

مثلاً Window = ۳۰ روز برای یک Bonus کوچک.

Explanation Recommendation ۱۰.۱۳

از Signal واقعی Reason می‌سازیم:

- «چون Science را انتخاب کرده‌ای.»
- «چون چند Idea درباره Focus ذخیره کرده‌ای.»
- «چون این Creator را دنبال می‌کنی.»

- «موضوع مشابه، اما این بار از یک «.Podcast

Reason باید با Score واقعی سازگار باشد.

۱۱.۱۳ Ranking Feed و Idea Related جدا هستند

۱.۱۱.۱۳ Feed

User-specific.

۲.۱۱.۱۳ Related

Idea-specific و کمتر وابسته به .User

این دو Service جدا طراحی می‌شوند.

۱۲.۱۳ ساختار کد پیشنهادی

```
apps/recommendations/├──
 scoring.py├──
 selectors.py├──
 explanations.py├──
 services.py└──
 tests/
```

۱۳.۱۳ Function Pure

مثال مفهومی:

```
def weighted_score(
    interest_match: float,
    saved_affinity: float,
    sparked_affinity: float,
    followed_creator: bool,
    popularity: float,
    freshness: float,
) -> float:
```

```
    score = 0.0
    score += 5 * interest_match
    score += 4 * saved_affinity
    score += 3 * sparked_affinity
    score += 3 if followed_creator else 0
    score += popularity
    score += freshness
    return score
```

در Implementation واقعی هاInput دقیق‌تر تعریف می‌شوند.

۱۴.۱۳ Performance

اشتباه:

```
ideas = Idea.objects.all()
# calculate every idea in Python for every request
```

بهتر:

۱. هاCandidate را با DB محدود کن؛
۲. filter؛ language/status/topic
۳. hidden/blocked؛ exclude
۴. window؛ candidate
۵. سپس .Score

۱۵.۱۳ Evaluation

هایTest لازم:

- start؛ cold
- ranking؛ interest
- adaptation؛ save/spark
- boost؛ follow
- filter؛ language
- penalty؛ seen
- diversity؛ cross-format
- result؛ deterministic

- latency زیر .Target

۱۶.۱۳ TF-IDF به‌عنوان Stretch

اگر similarity Topic-only ساده بود:

- TF-IDF روی ؛Title/Body
- Similarity؛ Cosine
- .scikit-learn

این قابلیت Optional است و بدون آن Project کامل محسوب می‌شود.

۱۷.۱۳ Conflict با Summarize in ۳۰s

Story منبع Chip Quick مثال Summarize in 30s دارد. چون AI Generative حذف شده:

- آن Story یا Deferred می‌شود؛
- یا به های‌Idea» کوتاه‌تر» تغییر می‌کند؛
- یا Excerpt Short دستی از قبل ذخیره می‌شود.

این تغییر Scope باید در گزارش نهایی صریح ذکر شود.

فصل ۱۴

Realtime، Sync Messaging و Notifications

۱.۱۴ چرا این بخش بعداً می‌آید؟

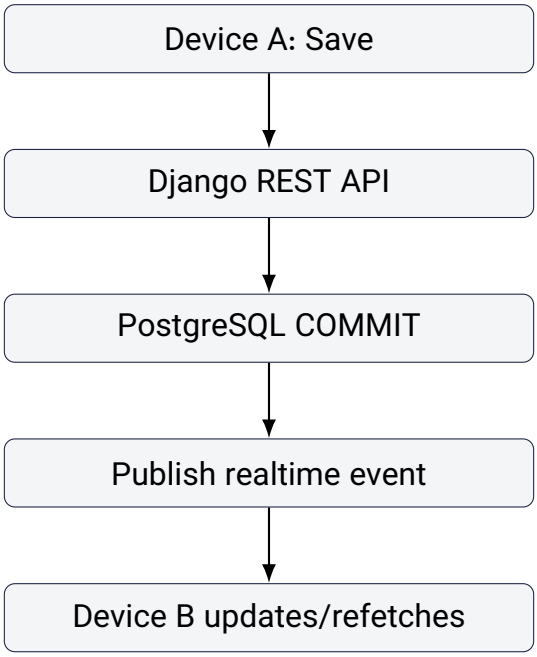
تا وقتی Save، User Library و Permission از طریق REST درست کار نکنند، WebSocket فقط Com-plexity اضافه می‌کند. بنابراین Realtime در Phase جدا وارد می‌شود.

۲.۱۴ Sync Cross-Device

نیازمندی:

- Save روی A؛ Device
- ظاهر شدن تغییر روی B Device ظرف حداکثر حدود ۲ ثانیه؛
- نمایش state. pending/synced

۳.۱۴ Flow صحیح Write



اگر Event fail delivery شد، Save اصلی هنوز در DB موجود است.

۴.۱۴ Channels و Redis

در این فاز:

- ASGI؛ Django
- Consumer؛ Channels
- Layer؛ Channel Redis
- WebSocket؛ authenticated
- handling. reconnect

۵.۱۴ Lifecycle WebSocket

Frontend باید بداند:

- connecting؛
- connected؛
- disconnected؛
- reconnecting؛

- failure. auth

۶.۱۴ Indicator Sync

UI می‌تواند:

- Saving...؛
- Saved؛
- pending؛ Sync
- Retry / failed Sync

نشان دهد.

۷.۱۴ Scope Messaging

تصمیم پروژه

Scope اولیه، Reader-to-Creator Messaging و Creator-to-Reader است؛ نه Messenger عمومی بین تمام کاربران.

۸.۱۴ Flow Message

۱. Client submit؛ message
۲. validation؛ authentication/participant
۳. Message save در PostgreSQL؛
۴. delivery realtime به participant؛ online
۵. UI update؛
۶. history همیشه از DB قابل بازیابی.

۹.۱۴ Authorization Conversation

هر Request برای Message باید بررسی کند:

- User authenticated است؛
- User participant همان Conversation است؛

- policy Block اجازه ارسال می‌دهد؛
- active Conversation است.

۱۰.۱۴ Indicator Typing

اگر اضافه شد، data Persistent نیست و می‌تواند فقط state transient realtime باشد.

برای messaging MVP ممکن است اصلاً اضافه نشود.

۱۱.۱۴ Receipt Read

Story منبع الزام صریحی به receipt read ندارد. بنابراین Optional است.

۱۲.۱۴ Block و Report

اگر User دیگری Block شود:

- ارسال پیام جدید باید محدود شود؛
- UI باید state را توضیح دهد؛
- rule server-side اجباری است.

Report conversation/message باید به workflow moderation وصل شود.

۱۳.۱۴ Notifications

types: Notification

- message؛ creator
- reply comment اگر later ؛added
- creators؛ followed from ideas new
- state sync در صورت نیاز؛
- outcome. moderation

۱۴.۱۴ Notification In-App اول

به دلیل support: old-device

تصمیم پروژه

baseline Notification In-App است. Push Web فقط بعد از test compatibility اضافه می‌شود.

۱۵.۱۴ Preference Notification

server-side: Model

```
NotificationPreference
- user
- type
- enabled
```

بنابراین Device جدید همان preference را می‌بیند.

۱۶.۱۴ Batching

story Source می‌خواهد ideas new از batch creators followed شوند.

راه ساده:

- های‌event مرتبط در بازه منطقی گروه شوند؛
- یک summary Notification ساخته شود.

اگر اجرای job periodic واقعاً لازم شد، آن زمان Queue Task بررسی می‌شود.

۱۷.۱۴ Test Realtime

session: دو Test

۱. login همان User در A و B؛
۲. save در A؛
۳. timestamp ثبت؛
۴. دریافت/update در B؛

۵. اندازه‌گیری latency؛
۶. چندین بار repeat.

۱۸.۱۴ Scenarios Failure

باید تست شود:

- Redis down؛
- WebSocket disconnect؛
- duplicate message attempt؛
- reconnect؛
- conversation unauthorized access؛
- user blocked send attempt.

۱۹.۱۴ تعریف Realtime Done

معیار پایان این مرحله

Realtime وقتی Done است که REST-only correctness حفظ شده باشد، reconnect درست کار کند، unauthorized event leakage وجود نداشته باشد و Sync latency چندبار اندازه‌گیری و ثبت شده باشد.

فصل ۱۵

PWA TTS، و سازگاری با گوشی‌های قدیمی

۱.۱۵ اصل سازگاری

بخش مهمی از کاربران فرضی ممکن است گوشی قدیمی داشته باشند. بنابراین experience Core نباید به های‌API جدید Browser وابسته باشد.

۲.۱۵ Tier بندی هایFeature

۱.۲.۱۵ Core :۱ Tier

باید تا حد ممکن روی های‌Browser قدیمی کار کند:

- login؛
- feed؛
- idea؛ read
- save؛
- spark؛
- comment؛
- library؛
- profile؛ creator
- search.

۲.۲.۱۵ Enhancement :۲ Tier

ممکن است روی Device قدیمی محدود باشد:

- install؛ PWA
- Push؛ Web
- caching؛ offline advanced
- features؛ voice TTS
- polish. realtime

۳.۱۵ چرا Web/PWA-first؟

Native های wrapper مدرن ممکن است حداقل iOS/Android جدیدتری بخواهند. app Web اجازه می‌دهد Core را روی های Browser قدیمی‌تر نیز ارزیابی کنیم.

۴.۱۵ PWA چیست؟

PWA ترکیبی از:

- Manifest؛ App Web
- Worker؛ Service
- installability در های Browser پشتیبانی‌شده؛
- behavior caching/offline محدود.

چیزی که باید یاد بگیری

PWA «تبدیل خودکار سایت به اپ Native کامل» نیست. قابلیت‌ها بسته به Browser/OS فرق می‌کنند.

۵.۱۵ Order Implementation PWA

۱. app web responsive کامل؛
۲. manifest؛
۳. icons؛
۴. worker؛ service
۵. assets؛ static cache
۶. fallback؛ offline
۷. فقط بعداً strategy cache برای API.

هشدار / دام رایج

API های response شخصی را بدون طراحی دقیق cache نکن؛ ممکن است stale data یا privacy bug ایجاد شود.

۶.۱۵ Requirement TTS

Source های story می‌خواهند:

- Play؛
- progress؛
- scrub؛
- selection. voice

۷.۱۵ گزینه TTS: Speech Browser/Device A

مزایا:

- بدون audio server generation؛
- بدون storage cost؛
- بدون AI/LLM؛
- استفاده از های Device. Voice

محدودیت:

- تفاوت شدید های Voice؛
- support Persian وابسته به Device؛
- seek/scrub دقیق سخت؛
- behavior متفاوت در iOS/Android.

۸.۱۵ گزینه TTS: Audio Pre-generated B

مزایا:

- فایل audio واقعی؛
- progress و scrub ساده‌تر؛
- behavior پایدارتر.

هزینه:

- provider TTS یا engine؛
- storage؛
- background generation؛
- media delivery؛
- complexity بیشتر.

۹.۱۵ تصمیم TTS

تصمیم پروژه

قبل از انتخاب A Option یا B، یک Prototype کوچک compatibility روی هایDevice قدیمی ساخته می‌شود. ساده‌ترین راهی که Requirement قابل قبول را تأمین کند انتخاب می‌شود.

۱۰.۱۵ Target اولیه Browser

هنوز Guarantee رسمی نداریم.

target: Evaluation

- iOS/Safari قدیمی در حدود iOS ۱۲+ در صورت دسترسی؛
- Android قدیمی در حدود Android ۷+؛
- Android فعلی؛
- Desktop؛ Chrome
- Desktop. Firefox

نتیجه واقعی در Matrix Compatibility ثبت می‌شود.

۱۱.۱۵ Performance Frontend روی Device ضعیف

قواعد:

- bundle JS کوچک؛
- splitting code در صورت نیاز؛
- تصاویر resize/compress؛

- image؛ load lazy
- animation محدود؛
- layers؛ blur/shadow expensive avoid
- pagination؛
- DOM؛ giant avoid
- autoplay. no

۱۲.۱۵ Build Legacy

وقتی هایDependency واقعی Frontend مشخص شدند:

۱. target Browser واقعی تست می‌شود؛
۲. missing هاsyntax/API شناسایی می‌شوند؛
۳. target/polyfill build حداقلی اضافه می‌شود.

هشدار / دام رایج

از روز اول انبوه polyfill نصب نمی‌کنیم. اول Evidence، بعد layer. Compatibility

۱۳.۱۵ Offline

Offline در سطح‌های متفاوت:

- shell؛ static
- idea؛ loaded previously
- snapshot؛ ideas saved
- download. eBook

برای MVP Core فقط installable/offline fallback کافی است. eBook offline story later phase است.

۱۴.۱۵ Matrix Compatibility

Device	Core	Install	TTS	Realtime	Notes
Android Old	TBD	TBD	TBD	TBD	تست واقعی

iPhone Old	TBD	TBD	TBD	TBD	تست واقعی
Current An- droid	TBD	TBD	TBD	TBD	
Chrome Desk- top	TBD	TBD	N/A	TBD	
Firefox Desk- top	TBD	TBD	N/A	TBD	

فصل ۱۶

بین‌المللی‌سازی، RTL/LTR و Accessibility

۱.۱۶ زبان‌ها

تصمیم پروژه

فارسی زبان اصلی و انگلیسی زبان دوم است. هر دو از ابتدای Frontend در معماری حضور دارند.

۲.۱۶ Language Content vs Language UI

دو مفهوم مستقل:

- preferred_ui_language
- preferences. language content

مثال:

- UI = فارسی؛
- content = فارسی + انگلیسی.

۳.۱۶ Files Translation

```
src/i18n/├──
 fa/│├──
    common.json│├──
    auth.json│├──
    feed.json│└──
```

```
    profile.json└──
 en/├──
     common.json├──
     auth.json├──
     feed.json└──
     profile.json
```

۴.۱۶ text Hard-coded

بعد از راه‌اندازی n:۱۸i

```
<button>{t("idea.save")}</button>
```

بهتر از:

```
<button>Save</button>
```

۵.۱۶ Direction

در document: Root

```
<html lang="fa" dir="rtl">
```

برای English:

```
<html lang="en" dir="ltr">
```

۶.۱۶ Properties Logical CSS

ترجیح:

```
margin-inline-start: 1rem;
padding-inline-end: 0.5rem;
text-align: start;
border-inline-start: 2px solid;
```

به جای هایleft/right زیاد.

۷.۱۶ Normalization Text Persian

Search فارسی باید به تفاوت‌های:

- ی/ي؛
- ک/ك؛
- نیم‌فاصله؛
- فاصله‌های اضافی؛
- punctuation

توجه کند.

هشدار / دام رایج

content Original را برای Search به‌صورت destructive تغییر نده. Normalization را برای representation Query/indexed انجام بده.

۸.۱۶ Target Accessibility

منبع WCAG ۲.۱ AA را می‌خواهد.

Project:

- target؛ minimum AA ۲.۱ WCAG
- استفاده از هایImprovement WCAG ۲.۲ در صورت ساده بودن؛
- عدم ادعای Compliance کامل بدون Audit واقعی.

۹.۱۶ HTML Semantic

قاعده:

- Button واقعی برای Action؛
- Link واقعی برای Navigation؛
- label؛ Form
- order؛ heading
- main/nav/section/article؛
- table. data برای table

۱۰.۱۶ Keyboard

تمام های‌Interaction مهم:

- Tab ؛reachable
- focus ؛visible
- behavior Enter/Space استاندارد؛
- trapping focus modal اگر modal وجود دارد؛
- action. hover-only no

۱۱.۱۶ Reader Screen

هاElement باید name accessible داشته باشند.

button: Icon-only

```
<button aria-label={t("idea.save")}>
  ...
</button>
```

ARIA فقط وقتی HTML Semantic کافی نیست.

۱۲.۱۶ Color/Contrast

theme dark prototype Design دارد. باید Contrast واقعی:

- متن اصلی؛
- متن ؛secondary
- ؛button
- ؛focus
- error

تست شود.

۱۳.۱۶ Targets Touch

روی موبایل های‌Button کوچک icon-only باید target لمس کافی داشته باشند، حتی اگر visual icon کوچک باشد.

۱۴.۱۶ TTS مستقل از Accessibility

Feature TTS مصرف محتواست، جای Reader Screen و semantic Accessibility را نمی‌گیرد.

۱۵.۱۶ Testing

- manual؛ keyboard
- tree؛ accessibility browser
- later؛ checks axe-style automated
- flows؛ critical Playwright
- inspection. visual RTL

۱۶.۱۶ Done of Definition Bilingual

هر :Screen

- translation؛ fa
- translation؛ en
- RTL/LTR؛
- overflow؛ text long
- format number/date اگر مرتبط؛
- labels. accessible

فصل ۱۷

امنیت، حریم خصوصی و Moderation

۱.۱۷ اصل کلی

Security فقط setting Production نیست؛ از Model و Permission شروع می‌شود.

۲.۱۷ Password

hashing password Django استفاده می‌شود.

ممنوع:

- password؛ plaintext
- crypto؛ custom
- log؛ password
- password در query .string

۳.۱۷ Authorization vs Authentication

- Authentication: چه کسی؟
- Authorization: چه اجازه‌ای؟

هر modifying Endpoint هر دو را بررسی می‌کند.

۴.۱۷ Ownership Object

مثال‌ها:

- User فقط own؛ profile
- Creator فقط own؛ idea
- User فقط own؛ comment delete
- participant فقط own؛ conversation
- Moderator طبق؛ permission
- Admin. unrestricted

۵.۱۷ CSRF

اگر auth Cookie/Session استفاده شود، protection CSRF باید درست نگه داشته شود.

هشدار / دام رایج

برای راحت شدن ،CSRF development را disable globally نکن.

۶.۱۷ CORS

CORS فقط هایOrigin لازم را Allow می‌کند. * در Production برای API authenticated انتخاب مناسبی نیست.

۷.۱۷ XSS

User content مانند Comment نباید به‌عنوان HTML raw اجرا شود.

در React به‌صورت پیش‌فرض escaping string کمک می‌کند؛ از dangerouslySetInnerHTML بدون sanitization پرهیز شود.

۸.۱۷ Upload File

اگر upload Avatar/Cover/eBook شد:

- validation؛ type

- limit؛ size
- filename/storage؛ safe
- authorization؛
- عدم اعتماد به .extension

۹.۱۷ Data Privacy

۱۲NFR- شامل:

- history؛ reading
- ideas؛ saved
- personalization.

Design باید امکان:

- export؛
- delete/anonymize؛
- minimization data

را بدهد.

۱۰.۱۷ Account Delete

قبل از Implementation باید Policy تعیین شود:

- هاComment delete یا ؟anonymize
- content Creator چه می‌شود؟
- Messages چطور حفظ/حذف می‌شوند؟
- data Audit/moderation چه می‌شود؟

این Policy باید در گزارش نهایی ثبت شود.

۱۱.۱۷ Privacy Message Direct

- HTTPS/WSS؛
- access؛ participant-only
- filtering؛ query

- test؛ permission
- browsing. moderator casual no

Encryption End-to-End در Scope نیست.

۱۲.۱۷ Payment

اگر payment Membership اجرا شد:

- provider؛ external
- PAN/CVV؛ raw no
- reference؛ provider
- verification؛ webhook
- machine. state status payment

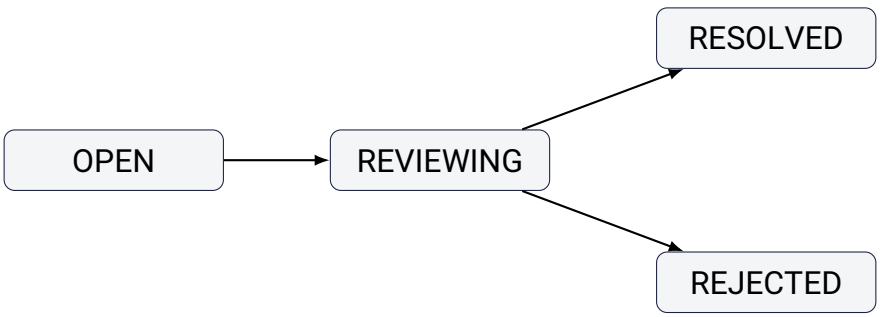
۱۳.۱۷ Report

Report های reason مشخص:

- harassment؛
- spam؛
- content؛ inappropriate
- category misinformation فقط اگر policy product مشخص شود؛
- other.

نباید از ابتدا سیاست‌های پیچیده و دسته‌بندی‌های زیاد بسازیم. دسته‌های نهایی Report در فاز Mod-eration مشخص می‌شوند.

۱۴.۱۷ Workflow Moderation



۱۵.۱۷ Audit ModerationAction

ثبت:

- actor؛
- target؛
- type؛ action
- reason/note؛
- timestamp.

۱۶.۱۷ Assignment Moderator

تصمیم پروژه

فقط Admin حق assign/remove کردن Moderator را دارد. این Rule باید Per- Server-side test mission داشته باشد.

۱۷.۱۷ Block

Block باید حداقل interaction Messaging را محدود کند.

۱۸.۱۷ Mute

Mute می‌تواند Notification/visibility را کاهش دهد بدون Block کامل.

Semantics نهایی هنگام implementation Feature نوشته می‌شود.

۱۹.۱۷ Limiting Rate

Candidate:

- login؛
- register؛
- comment؛

- report؛
- message؛
- search.

ساده‌ترین راه کافی انتخاب می‌شود.

۲۰.۱۷ Secrets

env. هرگز Commit نمی‌شود.

Secretها:

- key؛ secret Django
- password؛ DB
- later. credentials mail/payment

۲۱.۱۷ Logging

Log نکن:

- password؛
- token؛ auth
- data؛ sensitive payment
- body message private بی‌دلیل.

۲۲.۱۷ Feature Checklist Security

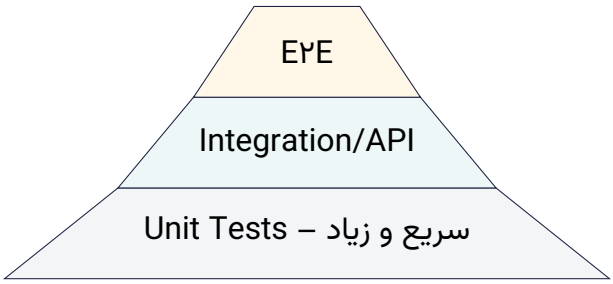
قبل از Done:

- test؛ request anonymous
- test؛ user wrong
- test؛ role wrong
- test؛ data invalid
- test؛ ownership
- check. leakage error

فصل ۱۸

Quality Testing، و سنجش هاNFR

۱.۱۸ Pyramid Test



۲.۱۸ Test Unit Backend

برای:

- scoring؛
- similarity؛
- helper؛
- function؛ validation
- normalization.

مثال:

```
def test_science_interest_ranks_science_idea_higher():
    ...
```

Test Model/DB ۳.۱۸

- save: unique
- spark: unique
- one-to-one: CreatorProfile
- constraints: status
- behavior. cascade/protect

Test API ۴.۱۸

سناریو:

```
anonymous save -> 401/403
authenticated save -> success
repeat save -> no duplicate
other user deletes comment -> forbidden
moderator hides comment -> success
moderator grants moderator -> forbidden
admin grants moderator -> success
```

Test Frontend ۵.۱۸

Library: Testing React

- text: rendered
- role/name: accessible
- interaction:
- loading/error:
- direction.

E۲E ۶.۱۸

:۱ Flow

```
register
-> choose language
-> choose interests
```

```
-> feed
```

:۲ Flow

```
login
-> open idea
-> spark
-> save
-> comment
-> library
```

:۳ Flow

```
creator page
-> follow
-> verify state
```

:۴ Flow

```
normal user cannot moderate
moderator can review
moderator cannot assign moderator
admin can assign moderator
```

۰۱NFR- ۷.۱۸

response perceived UI زیر ۱۰۰ms:

- update؛ state local
- Save/Spark؛ optimistic
- thread. main blocking avoid

latency API جدا اندازه‌گیری می‌شود.

۰۲NFR- ۸.۱۸

Feed page Next زیر ۳۰۰ms تحت condition test تعریف‌شده.

اندازه‌گیری:

- time؛ response server

- size؛ payload
- condition؛ network
- time. render

۰۳NFR- ۹.۱۸

Source «هزاران concurrent «user می‌خواهد.

برای پروژه دانشگاهی:

- progressive؛ test load
- ۵۰، ۱۰۰، ۵۰۰، ...؛
- report؛ bottleneck
- ادعای فقط همان چیزی که تست شده.

۰۴NFR- ۱۰.۱۸

path Design scaling Horizontal مستند می‌شود. لازم نیست Kubernetes بسازیم.

۰۵NFR- ۱۱.۱۸

latency: Sync

$$t_{receive} - t_{commit} \leq ۲s$$

چندین run و percentile بهتر از یک run موفق است.

۰۶NFR- ۱۲.۱۸

uptime ۹.۹۹% در student Project قابل اثبات سالانه نیست.

در گزارش:

- target؛ design
- assumption؛ deployment
- availability measured اگر period test داشتیم.

۱۳.۱۸ NFR-۰۷

resource: Idle

- devtools؛ browser
- monitor؛ system
- polling no بی‌دلیل؛
- animation. background no

۱۴.۱۸ NFR-۰۸

اگر CDN در Deployment واقعی اجرا شد Test می‌شود؛ در غیر این صورت limitation صریح ثبت می‌شود.

۱۵.۱۸ NFR-۱۶

Accessibility:

- keyboard؛
- checks؛ spot screen-reader
- audit؛ automated
- contrast.

۱۶.۱۸ NFR-۱۷

latency Recommendation زیر s.۲ چون LLM نداریم، باید بسیار کمتر هم قابل دستیابی باشد.

۱۷.۱۸ Data Test

seed: Deterministic

- User؛
- Creator؛
- Topic؛
- Source؛

- Idea؛
- Save/Spark/Follow؛
- Report.

۱۸.۱۸ CI

وقتی Test واقعی داریم:

- test؛ backend
- test؛ frontend
- lint؛
- build. frontend

CI قبل از وجود meaningful Test ارزش زیادی ندارد.

۱۹.۱۸ Coverage

Coverage کمک‌کننده است، نه .Goal

یک Permission مهم با ۱۰۰% coverage سطحی ممکن است هنوز Bug داشته باشد. Business -sce nario مهم‌تر است.

۲۰.۱۸ Gate Quality

قبل از milestone: Merge/Commit

- green؛ tests
- reasonable؛ lint
- secrets؛ no
- reviewed؛ migrations
- updated؛ docs
- test. smoke manual

فصل ۱۹

Git، Branching Workflow و نظم توسعه

۱.۱۹ هدف

Git فقط Backup نیست؛ تاریخچه تصمیم‌های پروژه است. برای پروژه یک‌نفره نیز باید قابل مرور باشد.

۲.۱۹ Phase ۰ Cleanup

قبل از Code اصلی:

۱. `git status`؛
۲. بررسی فایل‌های Track شده؛
۳. اضافه‌کردن `.gitignore`؛
۴. untrack کردن `.idea/` در صورت تصمیم نهایی؛
۵. حذف `main.py` starter در صورت تأیید؛
۶. تصمیم master/main Branch؛
۷. Commit. documentation

۳.۱۹ Strategy Branch

برای یک developer، workflow پیچیده GitFlow لازم نیست.

گزینه ساده:

- `main` یا `master` = stable؛ branch
- branch feature برای کارهای چندمرحله‌ای در صورت نیاز.

مثال:

```
feature/content-models
feature/feed-api
feature/react-feed
feature/recommendation
```

۴.۱۹ Message Commit

Format پیشنهادی:

```
type(scope): short description
```

Type:

- feat
- fix
- test
- docs
- refactor
- chore.

مثال:

```
feat(content): add Topic, Source and Idea models
feat(auth): add custom user model
test(engagement): prevent duplicate saves
docs(roadmap): document realtime phase
fix(i18n): correct RTL spacing in profile screen
```

۵.۱۹ قبل از Commit

۱. `git status`
۲. `git diff`
۳. tests
۴. secret no check
۵. migration check
۶. changed contract if documentation

۶.۱۹ های Commit بد

```
update
stuff
fix
final
final2
changes
```

این پیام‌ها بعداً هیچ یContext نمی‌دهند.

۷.۱۹ Migration و Commit

change Model و Migration مربوط باید در همان Commit باشند مگر دلیل واضح دیگری وجود داشته باشد.

۸.۱۹ Lockfile

lockfile Frontend باید Commit شود تا reproducible resolution dependency باشد.

strategy dependency Python نیز باید reproducible باشد.

۹.۱۹ Check Secrets

قبل از :Push

- env. نباشد؛
- key secret نباشد؛
- password DB نباشد؛
- later credential payment/email نباشد.

۱۰.۱۹ های Milestone Tag

در پایان هایmilestone مهم:

```
v0.1-core-backend
```

```
v0.2-core-feed
v0.3-engagement
v0.4-recommendation
v1.0-bachelor-release
```

naming Tag نهایی می‌تواند ساده‌تر باشد.

۱۱.۱۹ Journal Project

یک فایل مثل:

```
docs/devlog/
  2026-xx-xx-phase-01.md
```

اختیاری اما مفید است.

محتوا:

- learned؛
- changed؛
- bug؛
- decision؛
- next.

۱۲.۱۹ Mindset Recovery

قبل از Command خطرناک Git:

- وضعیت را بخوان؛
- بفهم Command چه می‌کند؛
- از hard reset– بدون نیاز استفاده نکن؛
- work uncommitted را بی‌دلیل حذف نکن.

فصل ۲۰

Roadmap اجرایی کامل پروژه

۱.۲۰ قاعده Roadmap

هر فاز:

$$Learn \rightarrow Build \rightarrow Test \rightarrow Document \rightarrow Commit$$

نباید به‌خاطر جذاب بودن یک فناوری، order Dependency شکسته شود.

۲.۲۰ Phase ۰: Foundation Repository

یادگیری

- Git: status/diff/add/commit
- virtual: environment
- dependency:
- monorepo: concept
- variable. env

ساخت

- repo: cleanup
- .gitignore:
- backend/frontend: directories
- PostgreSQL: ready
- dependency: strategy

- commit. documentation

معیار پایان این مرحله

Environment قابل بازسازی است و structure Root تمیز است.

۳.۲۰ Phase ۱: Foundation Django

یادگیری

Project/App/Settings/URL/View/Model/Migration/ORM/Admin.

ساخت

- project Django در backend؛
- config؛ PostgreSQL
- User؛ custom
- endpoint؛ health
- superuser. admin

معیار پایان این مرحله

Django با PostgreSQL اجرا می‌شود و اولین Migration با User Custom ثبت شده است.

۴.۲۰ Phase ۲: Models Content Core

یادگیری

index. constraint، ManyToMany، OneToOne، ForeignKey،

ساخت

- CreatorProfile؛
- Topic؛
- UserInterest؛
- Source؛
- Idea؛

- config؛ Admin
- data. demo

۵.۲۰ Phase ۳: Foundation API REST

یادگیری

pagination. status، HTTP APIView/generic/ViewSet، validation، Serializer،

ساخت

- API؛ Topic
- API؛ Idea
- API؛ read Creator
- baseline؛ feed
- tests.

۶.۲۰ Phase ۴: Roles and Authentication

یادگیری

ownership. object group/permission، authorization، vs Auth

ساخت

- register/login/logout؛
- profile؛
- permission؛ Creator
- group؛ Moderator
- assignment؛ moderator Admin-only
- tests.

۷.۲۰ Phase ۵: Fundamentals Frontend

یادگیری

HTML، CSS، Browser JS، npm، TypeScript. Vite

ساخت

- scaffold: React-TS
- tokens: theme
- layout: mobile static
- foundation. RTL/LTR

۸.۲۰ Phase ۶: Foundation React

یادگیری

Component، props، state، hooks. routing

ساخت

- routing:
- IdeaCard:
- states: common
- skeleton: client API
- switch. language

۹.۲۰ Phase ۷: Onboarding

- intro:
- integration: register/login
- selection: language
- selection: interest
- interests: persist
- redirect. feed first

۱۰.۲۰ Phase ۸: Feed Core

- endpoint؛ paginated
- query؛ infinite
- data؛ real IdeaCard
- Detail؛ Idea
- display؛ Source
- estimate. reading

۱۱.۲۰ Phase ۹: Engagement

- API/UI؛ Spark
- API/UI؛ Save
- update؛ optimistic
- Comment؛
- comment؛ report
- share. external

۱۲.۲۰ Phase ۱۰: Library Personal

- Ideas؛ Saved
- filters؛
- statistics reading فقط بعد از تعریف ؛read
- data. foundation cross-device

۱۳.۲۰ Phase ۱۱: Network Creator

- Profile؛ Creator
- ideas؛ Creator
- Follow؛
- counts. follower/spark/idea

۱۴.۲۰ Phase ۱۲: Search and Explore

- search، query
- topics،
- filters، format
- normalization، Persian
- definition trending ساده.

۱۵.۲۰ Phase ۱۳: Algorithm Recommendation

- start، cold
- score، interest
- affinity، save/spark
- signal، follow
- freshness/popularity،
- fit، language
- ideas، related Jaccard
- explanation،
- evaluation.

۱۶.۲۰ Phase ۱۴: Sync Realtime

در اینجا برای اولین بار:

- Redis،
- Channels،
- WebSocket،
- reconnect،
- indicator. sync

۱۷.۲۰ Phase ۱۵: Messaging

- Conversation/Participant/Message،
- rules، reader-creator
- history، persisted

- delivery؛ realtime
- block/report.

۱۸.۲۰ Phase ۱۶: Notifications

- center؛ in-app
- preferences؛
- batching؛
- later. feasibility push

۱۹.۲۰ Phase ۱۷: TTS

۱. prototype؛ compatibility
۲. audio؛ generated vs browser compare
۳. acceptable؛ simplest choose
۴. controls؛ implement
۵. Persian/English. test

۲۰.۲۰ Phase ۱۸: Membership/eBooks

- metadata؛ eBook
- state؛ membership
- entitlement؛
- requires؛ scope final if only payment
- feasible. if sample/feature offline

۲۱.۲۰ Phase ۱۹: Completion Moderation

- queue؛ report
- action؛ moderation
- audit؛
- restrictions؛ user
- management. moderator Admin

۲۲.۲۰ Phase ۲۰: Hardening

- accessibility؛
- test؛ old-device
- performance؛
- review؛ security
- indexes؛
- polish. bilingual

۲۳.۲۰ Phase ۲۱: Deployment

- settings؛ production
- env؛
- PostgreSQL؛
- HTTPS؛
- static/media؛
- logs/backup.

۲۴.۲۰ Phase ۲۲: Package Academic Final

- diagrams؛ architecture final
- ERD؛
- screenshots؛
- evidence؛ test
- evaluation؛ NFR
- limitations؛
- guide؛ installation
- report/presentation. final

۲۵.۲۰ Rules Dependency

هشدار / دام رایج

نمونه هایJump ممنوع:

- WebSocket قبل از Save؛ REST
- Recommendation قبل از Topic/Idea؛ data
- Payment قبل از ؛Library
- caching PWA پیچیده قبل از responsive؛ app
- Redux قبل از مشاهده مشکل management. state

فصل ۲۱

های‌Playbook پیاده‌سازی Feature

این فصل می‌گوید وقتی AI در دسترس نیست و می‌خواهی Feature جدید بسازی، چه ترتیب ثابتی را طی کنی.

۱.۲۱ Playbook ساخت Model جدید

فرض: می‌خواهی Model جدید بسازی.

۱. Requirement را بنویس.
۲. Entity و Relation را روی کاغذ مشخص کن.
۳. آیا Model جدید واقعاً لازم است؟
۴. هاField و data هاtype را تعیین کن.
۵. unique/check/index را مشخص کن.
۶. ForeignKey behavior deletion را تعیین کن.
۷. code Model بنویس.
۸. `makemigrations`.
۹. Migration را بخوان.
۱۰. `migrate`.
۱۱. query. shell Django
۱۲. test. Model
۱۳. Admin در صورت نیاز.
۱۴. Commit.

۲.۲۱ Playbook ساخت API جدید

۱. case Use را دقیق بنویس.
۲. Method/path را تعیین کن.
۳. params. body/query Request
۴. Authentication.
۵. ownership. Permission/object
۶. Serializer/validation.
۷. Query/Service.
۸. schema. Response
۹. codes. status
۱۰. test. happy-path
۱۱. test. invalid-data
۱۲. test. unauthorized
۱۳. test. object-ownership
۱۴. document.

۳.۲۱ Playbook ساخت Screen React

۱. Route؛
۲. requirement؛ data
۳. state؛ loading
۴. state؛ empty
۵. state؛ error
۶. markup؛ semantic static
۷. CSS؛ responsive
۸. RTL/LTR؛
۹. split؛ component
۱۰. integration؛ API
۱۱. interaction؛
۱۲. accessibility؛
۱۳. tests.

۴.۲۱ Mutation Playbook مثل Save

۱. endpoint server را اول درست کن؛
۲. idempotency؛
۳. auth/permission؛
۴. constraint؛ DB
۵. test؛ API
۶. button؛ frontend
۷. mutation؛
۸. state؛ optimistic
۹. rollback؛
۱۰. invalidation/reconciliation؛ query
۱۱. later. event cross-device

۵.۲۱ Pagination Playbook

۱. ordering؛ stable
۲. method؛ pagination
۳. size؛ page
۴. tests؛ endpoint
۵. page؛ next frontend
۶. prevention؛ duplicate
۷. indicator؛ loading
۸. state؛ no-more-results
۹. measurement. performance

۶.۲۱ Permission Playbook

برای هر Action جدول بساز:

Action	Reader	Creator	Moderator	Admin
idea public Read	بله	بله	بله	بله
idea creator own Edit	–	بله	policy	بله
comment reported Hide	–	–	بله	بله

moderator Assign	–	–	بله

سپس Test همان جدول را بنویس.

۷.۲۱ Recommendation Playbook تغییر Weight

۱. Hypothesis؛
۲. scenario؛ fixture
۳. ranking؛ old
۴. change؛ weight
۵. tests؛ unit
۶. check؛ diversity
۷. check؛ latency
۸. reason. document

۸.۲۱ Bug Playbook

۱. reproduce؛
۲. case؛ minimal
۳. log/inspect؛
۴. layer؛ locate
۵. test failing write اگر ممکن؛
۶. fix؛
۷. tests؛ related rerun
۸. check؛ regression
۹. explanation. with commit

۹.۲۱ Performance Database Playbook

۱. endpoint؛ slow reproduce
۲. measure؛
۳. queries؛ count
۴. ۱N+ inspect؛
۵. `select\_related/prefetch\_related`؛

۶. plan؛ index/query inspect
۷. remeasure.

Cache آخرین پاسخ پیش‌فرض نیست.

۱۰.۲۱ Dependency Playbook

قبل از نصب :Package

۱. Requirement؛
۲. alternatives؛
۳. maintenance؛
۴. size/compatibility؛
۵. security؛
۶. install؛
۷. example؛ minimal
۸. integrate؛
۹. document.

۱۱.۲۱ Refactor Playbook

Refactor یعنی Behavior را حفظ کن:

۱. before؛ green tests
۲. change؛ small
۳. mixed؛ addition feature no
۴. after؛ green tests
۵. review. diff

۱۲.۲۱ Playbook وقتی گیر کردی

بدون :AI

۱. error را کامل بخوان؛
۲. traceback از آخر به اول؛
۳. کوچک‌ترین component؛ failing

۴. docs؛ official
۵. input/output؛ inspect
۶. shell/test؛ in reproduce
۷. تغییر یک چیز در هر بار.

هشدار / دام رایج

ده Package نصب نکن تا Bug «شاید» حل شود. اول علت را پیدا کن.

فصل ۲۲

Deployment و Readiness Production

۱.۲۲ هدف

Deployment این پروژه باید ساده و قابل دفاع باشد، نه نمایش .DevOps

۲.۲۲ های Production Component

حداقل:

- build؛ static Frontend
- API؛ Django
- PostgreSQL؛
- HTTPS؛
- handling. media/static

بعداً اگر :Realtime

- server؛ ASGI
- Redis.

۳.۲۲ Build Frontend

```
npm run build
```

خروجی asset Static باید روی Host/CDN مناسب سرو شود.

۴.۲۲ Settings Production Django

- DEBUG=False؛
- SECRET_KEY از env؛
- ALLOWED_HOSTS؛
- cookies؛ secure
- settings؛ HTTPS
- config؛ production DB
- config؛ static/media
- origin. CORS/CSRF

۵.۲۲ ASGI vs WSGI

بدون Django realtime، می‌تواند با WSGI/ASGI مناسب اجرا شود.

با ASGI Channels/WebSocket، لازم است.

باید تفاوت را هنگام فاز Realtime یاد بگیری.

۶.۲۲ Static و Media

- assets؛ CSS/JS/build = Static
- files. uploaded user/content = Media

این دو را مخلوط نکن.

۷.۲۲ Deployment Migration Database

Flow:

۱. backup؛
۲. code؛ deploy
۳. migrate؛
۴. test. smoke

destructive Migration باید قبل از production بررسی شود.

۸.۲۲ Backup

حداقل:

- strategy؛ dump PostgreSQL
- backup media اگر مهم؛
- test restore در صورت امکان.

Backup بدون تست Restore اطمینان کامل نمی‌دهد.

۹.۲۲ Logging

log: Production

- error؛ request/server
- warning؛
- needed. where moderation/audit

content Sensitive کمینه.

۱۰.۲۲ Monitoring

برای پروژه دانشگاهی:

- logs؛ application
- observation؛ uptime basic
- health؛ database
- tracking error اگر سرویس ساده در دسترس بود.

نباید stack monitoring سنگین بسازیم.

۱۱.۲۲ CDN

-NFR۰۸ CDN می‌خواهد.

اگر Deployment واقعی امکان CDN داشت:

- CDN؛ static/media
- headers؛ cache

- versioning. asset

اگر نداشت:

> **هشدار / دام رایج**
>
> در گزارش نهایی صریحاً بنویس NFR-۰۸ به‌طور کامل در محیط دانشگاهی Realize نشده است؛ به‌جای ادعای ساختگی.

۱۲.۲۲ Path Scaling

اگر روزی Load زیاد شد:

- چند Django instance؛
- proxy/load reverse balancer؛
- DB tuning؛
- cache؛
- Redis؛
- jobs. background

این‌ها path Future هستند.

۱۳.۲۲ Checklist Release

- tests pass؛
- migrations applied؛
- DEBUG off؛
- secrets absent from repo؛
- HTTPS؛
- access admin secured؛
- accounts sample removed؛
- static/media reachable؛
- smoke bilingual test؛
- backup taken؛
- E.۲E flow critical

فصل ۲۳

ردیابی کامل نیازمندی‌های منبع

۱.۲۳ هدف

این فصل برای این است که در پایان پروژه بتوانیم نشان دهیم هر Story و NFR از کجا آمده، چگونه تفسیر شده و آیا پیاده‌سازی شده، ساده شده یا Deferred است.

۲.۲۳ Discovery Idea AI-Powered Epic:

۱. به‌عنوان Reader می‌خواهم هایIdea مشابه Idea فعلی را ببینم تا روی Theme موردعلاقه عمیق‌تر شوم. [NFR-17]
۲. می‌خواهم یک دلیل یک‌خطی برای پیشنهاد ببینم تا Recommendation قابل اعتماد باشد. [NFR-17]
۳. می‌خواهم Chip Suggestion Quick داشته باشم، مثل Summarize in 30s، تا سریع‌تر Action بگیرم. [NFR-01]
۴. می‌خواهم Feed Home به هایInterest اعلام‌شده نزدیک باشد. [NFR-01]
۵. می‌خواهم Feed با Spark و هایSave من adapt شود. [NFR-12]
۶. می‌خواهم suggestion Cross-format داشته باشم؛ مثلاً Idea Book به Podcast مرتبط برسد. [NFR-03]
۷. به‌عنوان User جدید می‌خواهم در Topic Onboarding انتخاب کنم. [NFR-01]
۸. می‌خواهم هاInterest را بعداً از Profile تغییر بدهم. [NFR-12]

تصمیم پروژه

تفسیر پروژه: Recommendation کاملاً الگوریتمی است. «Summarize Story in ۳۰s» gener- ative پیاده نمی‌شود مگر با Content Short از پیش نوشته‌شده یا جایگزین غیرمولد.

Experience Cross-Device Real-Time، Epic: ۳.۲۳

۱. Save روی Phone ظرف چند ثانیه در Library Laptop دیده شود. [NFR-05]
۲. indicator synced/pending وجود داشته باشد. [NFR-05]
۳. card Idea و thumbnail در دنیا بدون delay محسوس load شود. [NFR-08]
۴. روی connection کند، audio TTS سریع شروع شود. [NFR-08]
۵. Backend در scale spike traffic شود. [NFR-04]
۶. App در available hour peak بماند. [NFR-06]

Network Direct-Access Creator Epic: ۴.۲۳

۱. Reader پروفایل ،Bio Creator و Ideas Published را ببیند. [NFR-01]
۲. Creator Reader را Follow کند. [NFR-01]
۳. reach Creator خود شامل Idea/Spark/Follower را ببیند/نمایش دهد. [NFR-01]
۴. Reader بتواند Creator را مستقیم Message کند. [NFR-13]
۵. Creator داخل Reply App بدهد. [NFR-13]
۶. User بتواند در Report/Block Conversation کند. [NFR-15]
۷. Creator غیرانگلیسی‌زبان در زبان خودش discoverable باشد. [NFR-18]

eBooks & Membership Library Epic: ۵.۲۳

۱. تمام Saved هاIdea در tab Library باشند. [NFR-05]
۲. تعداد هایIdea خوانده‌شده ماه دیده شود. [NFR-11]
۳. هایeBook مرتبط با هایIdea موردعلاقه browse شوند. [NFR-11]
۴. membership Library امن purchase شود. [NFR-14]
۵. Member بتواند eBook را download offline کند. [NFR-11]

Content Multi-Format & Feed Idea Core Epic: ۶.۲۳

۱. paginated Feed باشد. [NFR-02]
۲. time reading/listening Estimated روی هر card باشد. [NFR-01]
۳. text full Idea باز شود. [NFR-01]
۴. source Original مشخص باشد. [NFR-08]
۵. Reader تجربه مرتبط با Book/Movie/Podcast/Article را Share کند. [NFR-08]
۶. Idea خارج از Share App شود. [NFR-08]

۷.۲۳ Setup Account & Onboarding Epic:

۱. Account سریع با Email یا Sign-in Social ساخته شود. [NFR-09]
۲. returning User روی Device جدید Library قبلی را ببیند. [NFR-05]
۳. Intro کوتاه هدف InsightInk را توضیح دهد. [NFR-01]

۸.۲۳ Signals Social & Engagement Epic:

۱. Reader بتواند Idea را Spark کند. [NFR-01]
۲. Idea Reader را Save کند. [NFR-05]
۳. Comment Reader بگذارد. [NFR-01]
۴. Comment Reader نامناسب را Report کند. [NFR-15]

۹.۲۳ (Text-to-Speech) Access Multimodal Epic:

۱. Play و شنیدن .Idea [NFR-08]
۲. progress playback و .scrub [NFR-16]
۳. انتخاب voice. TTS [NFR-16]
۴. navigation reader Screen بدون .Mouse [NFR-16]

۱۰.۲۳ Awareness Activity & Notifications Epic:

۱. وضعیت Save Sync دیده شود. [NFR-05]
۲. هایIdea جدید هایFollow Creator شده batch شوند. [NFR-06]
۳. User بتواند Notification هایtype خاص را خاموش کند. [NFR-06]

۱۱.۲۳ جدول NFR Treatment

ID	Source	Treatment پروژه
NFR-۰۱	UI <۱۰۰ms	UI optimistic + target UI perceived
NFR-۰۲	page feed <۳۰۰ms	target API/client measurable

NFR-۰۳	concur- thousands rent	evidence- testing، load progressive claim based
NFR-۰۴	scale elastic	base- Kubernetes نه path، architecture line
NFR-۰۵	s۲> sync	measurement + later Channels/Redis
NFR-۰۶	uptime %۹.۹۹	target، design نه SLA ادعایی
NFR-۰۷	CPU/RAM idle low	measurement device/browser
NFR-۰۸	CDN	limitation deployment-dependent؛ در صورت عدم اجرا
NFR-۰۹	native +۹+/Android۱۴iOS	strategy old-device web/PWA-first
NFR-۱۰	PWA installable	supported where retained
NFR-۱۱	SQL relational	PostgreSQL
NFR-۱۲	export/delete per- data sonal	design lifecycle
NFR-۱۳	DM secure/participant-only	authorization + HTTPS/WSS
NFR-۱۴	card raw PCI/no	only provider external
NFR-۱۵	report/block/mute	retained
NFR-۱۶	AA ۱.۲ WCAG	target accessibility minimum
NFR-۱۷	s۲> AI	latency recommendation algorithmic
NFR-۱۸	multilingual	one day from English + Persian

۱۲.۲۳ WSJF کامل

Epic	BV	TC	RR/OE	Effort	WSJF
Discovery Idea AI-Powered	۶	۵	۵	۷	۲۸.۲
Cross-Device Real-Time	۸	۷	۸	۵	۶۰.۴
Direct-Access Creator	۶	۳	۸	۵	۴۰.۳

Membership/eBooks Library	۸	۷	۸	۴	۷۵.۵
Feed/Multi-Format Core	۷	۸	۸	۶	۸۳.۳
Onboarding/Account	۶	۸	۱۰	۳	۰۰.۸
Signals Engagement/Social	۴	۳	۴	۵	۲۰.۲
TTS/Multimodal	۶	۲	۷	۴	۷۵.۳
Notifications	۳	۲	۵	۴	۵۰.۲

Reduction/Opportunity Risk = RR/OE Criticality، Time = TC Value، User/Business = BV Enablement.

۱۳.۲۳ پیشنهادی برای پایان پروژه Matrix Traceability

در پایان، جدول زیر تکمیل شود:

Story/NFR	Implementation	Status	Evidence
US-...	endpoint یا screen	وضعیت اجرا	test یا screenshot یا commit

این Matrix یکی از مهم‌ترین خروجی‌ها برای گزارش دانشگاهی است.

فصل ۲۴

تصمیم‌های باز و راهنمای ادامه Cheat، Sheet بدون AI

۱.۲۴ هایCommand پایه Git

```
git status
git diff
git add <file>
git commit -m "feat(scope): message"
git log --oneline --decorate --graph
git branch
```

۲.۲۴ Python/Virtualenv

```
.venv/bin/python --version
.venv/bin/python -m pip --version
.venv/bin/python -m pip list
```

۳.۲۴ Django

بعد از Scaffold:

```
cd backend

../.venv/bin/python manage.py check
../.venv/bin/python manage.py makemigrations
```

```
../.venv/bin/python manage.py migrate
../.venv/bin/python manage.py createsuperuser
../.venv/bin/python manage.py runserver
../.venv/bin/python manage.py shell
../.venv/bin/python manage.py test
```

وقتی pytest اضافه شد:

```
../.venv/bin/python -m pytest
```

۴.۲۴ Frontend

بعد از Scaffold:

```
cd frontend

npm install
npm run dev
npm run build
npm run test
```

هایScript واقعی از `package.json` خوانده شوند.

۵.۲۴ Checklist Conceptual PostgreSQL

باید بتوانی:

- list؛ database
- create؛ database
- concept؛ user/role
- connect؛
- SELECT؛ basic
- table؛ inspect
- concept. backup/restore

هاCommand بسته به setup سیستم در Phase ۰ ثبت می‌شوند.

۶.۲۴ Reference Quick HTTP

Code	Meaning	Example
۲۰۰	OK	success GET
۲۰۱	Created	created resource
۲۰۴	Content No	response delete/toggle
۴۰۰	Request Bad	validation
۴۰۱	Unauthorized	authenticated not
۴۰۳	Forbidden	allowed not but authenticated
۴۰۴	Found Not	visible missing/not resource
۴۰۹	Conflict	used if conflict state
۵۰۰	Error Server	failure backend unhandled

۷.۲۴ Backend Checklist Debugging

۱. traceback؛ full
۲. traceback؛ in file project first
۳. data؛ input
۴. state؛ DB
۵. errors؛ serializer
۶. permission؛
۷. count؛ query
۸. test/shell. in reproduce

۸.۲۴ Frontend Checklist Debugging

۱. console؛ browser
۲. tab؛ network
۳. URL/method/body؛ request
۴. status/body؛ response
۵. props؛ component
۶. cache؛ state/query

۷. style؛ computed RTL/CSS
۸. minimal. reproduce

۹.۲۴ Name Repository Decision: Open

folder: Current

```
InkSight
```

Product:

```
InsightInk
```

قبل از شروع جدی بهتر است تصمیم rename گرفته شود.

۱۰.۲۴ Branch Git Decision: Open

Current:

```
master -> origin/master
```

می‌توان نگه داشت یا به main تغییر داد. فقط آگاهانه.

۱۱.۲۴ Management Dependency Python Decision: Open

انتخاب نهایی:

- ساده: requirements.txt؛
- مدرن: pyproject.toml + tool.

برای پروژه آموزشی، هر کدام که انتخاب شد باید تا پایان consistent بماند.

۱۲.۲۴ Login Social Decision: Open

Source درخواست email یا sign-in social دارد.

گزینه:

- deferred؛ social + core email

- یک social provider برای تکمیل .Story

۱۳.۲۴ Definition Read Decision: Open

برای stats Reading باید دقیقاً تعریف شود:

- read؟ = open
- time؟ dwell
- complete؟ explicit
- completion؟ TTS

۱۴.۲۴ TTS Decision: Open

بعد از prototype. compatibility

۱۵.۲۴ Payment/eBooks Decision: Open

آیا payment submission bachelor Final واقعی می‌خواهد یا membership Demo کافی است؟

۱۶.۲۴ Push Web Decision: Open

بعد از test. old-device

۱۷.۲۴ وقتی AI در دسترس نیست

ترتیب پیدا کردن جواب:

۱. همین سند؛
۲. code/tests موجود؛
۳. docs؛ Django/DRF/React/Vite/PostgreSQL official
۴. error/traceback؛
۵. reproduction؛ minimal
۶. issue search فقط بعد از فهم .error

۱۸.۲۴ قاعده مطالعه Documentation

وقتی Docs Official می‌خوانی:

- version همان dependency را چک کن؛
- tutorial و reference را تفکیک کن؛
- example را کورکورانه copy نکن؛
- با context project تطبیق بده.

۱۹.۲۴ چک‌لیست شروع هر Feature

۱. Story/NFR چیست؟
۲. model Data چه می‌خواهد؟
۳. endpoint Backend چیست؟
۴. Permission چیست؟
۵. state Frontend چیست؟
۶. فارسی/انگلیسی چه می‌شود؟
۷. fallback old-device چیست؟
۸. test چیست؟
۹. criteria Done چیست؟

۲۰.۲۴ چک‌لیست پایان هر Session

۱. run؛ tests
۲. status؛ git
۳. diff؛ git
۴. secret؛ no
۵. update؛ docs
۶. commit؛ meaningful
۷. note. task next

۲۱.۲۴ Deliverables Project Final

در پایان باید داشته باشیم:

- code؛ source
- guide؛ setup
- ERD؛ database
- docs؛ API
- diagram؛ architecture
- traceability؛ requirement
- tests؛
- matrix؛ compatibility
- results؛ performance/NFR
- screenshots/demo؛
- report/PDF؛ LaTeX final
- presentation.

۲۲.۲۴ آخرین اصل

هیچ بخش مهم پروژه نباید فقط به این دلیل وجود داشته باشد که AI آن را پیشنهاد داده است.

هر تصمیم باید ،Requirement دلیل فنی و مسیر Test داشته باشد.

جمع‌بندی

معماری این پروژه عمداً طوری طراحی شده که برای یک پروژه کارشناسی واقعی، قابل‌اتمام و قابل‌فهم باشد. هسته سیستم یک Django modular monolith با پایگاه داده PostgreSQL و یک رابط React + TypeScript + Vite است. قابلیت‌های سنگین‌تر مانند WebSocket/Redis فقط در زمانی وارد می‌شوند که Story مربوط به همگام‌سازی یا پیام‌رسانی پیاده‌سازی شود.

مهم‌ترین اصل پروژه این است که هیچ فناوری صرفاً برای «مدرن به نظر رسیدن» اضافه نشود. هر ابزار باید یک نیاز مشخص را حل کند و توسعه‌دهنده بتواند آن را توضیح دهد.